

BACK PROPAGATION

- Review of Multi Layer Structures
- Problem Definition
- Calculation of Error Gradients
- Back Propagation Algorithm
- Momentum Method
- Various Problems/Modifications
- Neural Networks as Universal Approximators
- Structure Selection

- **Neural Networks As Universal Function Approximators**

- Approximation problem ...

$$h : \mathbf{R}^n \rightarrow \mathbf{R}^m$$

$$\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$$

Training Set



x



$h(x)$



$$\{h(x^1), h(x^2), \dots, h(x^N)\}$$

Desired output (Sampling)



$$o(x) = H(W, x)$$

• The learning problem is : Find a weight W_* such that $H(W_*, x)$ is as close to the $h(x)$ over the training set as possible.

- **ALWAYS POSSIBLE WITH NEURAL NETWORKS**

training set $\mathcal{T}_r = \{x^1, x^2, \dots, x^N\}$

For pattern x^i , the approximation error : $E_i = \|H(W, x^i) - h(x^i)\|^2$

L_2 **norm**

training set $\mathcal{T}_r = \{x^1, x^2, \dots, x^N\}$

For pattern x^i , the approximation error : $E_i = \|H(W, x^i) - h(x^i)\|^2$

L_2 **norm**

$$E(W) = \frac{1}{N} \sum_{i=1}^N E_i = \frac{1}{N} \sum_{i=1}^N \|H(W, x^i) - h(x^i)\|^2$$

Average error over training set

training set $\mathcal{T}_r = \{x^1, x^2, \dots, x^N\}$

For pattern x^i , the approximation error : $E_i = \|H(W, x^i) - h(x^i)\|^2$

L_2 norm

$$E(W) = \frac{1}{N} \sum_{i=1}^N E_i = \frac{1}{N} \sum_{i=1}^N \|H(W, x^i) - h(x^i)\|^2$$

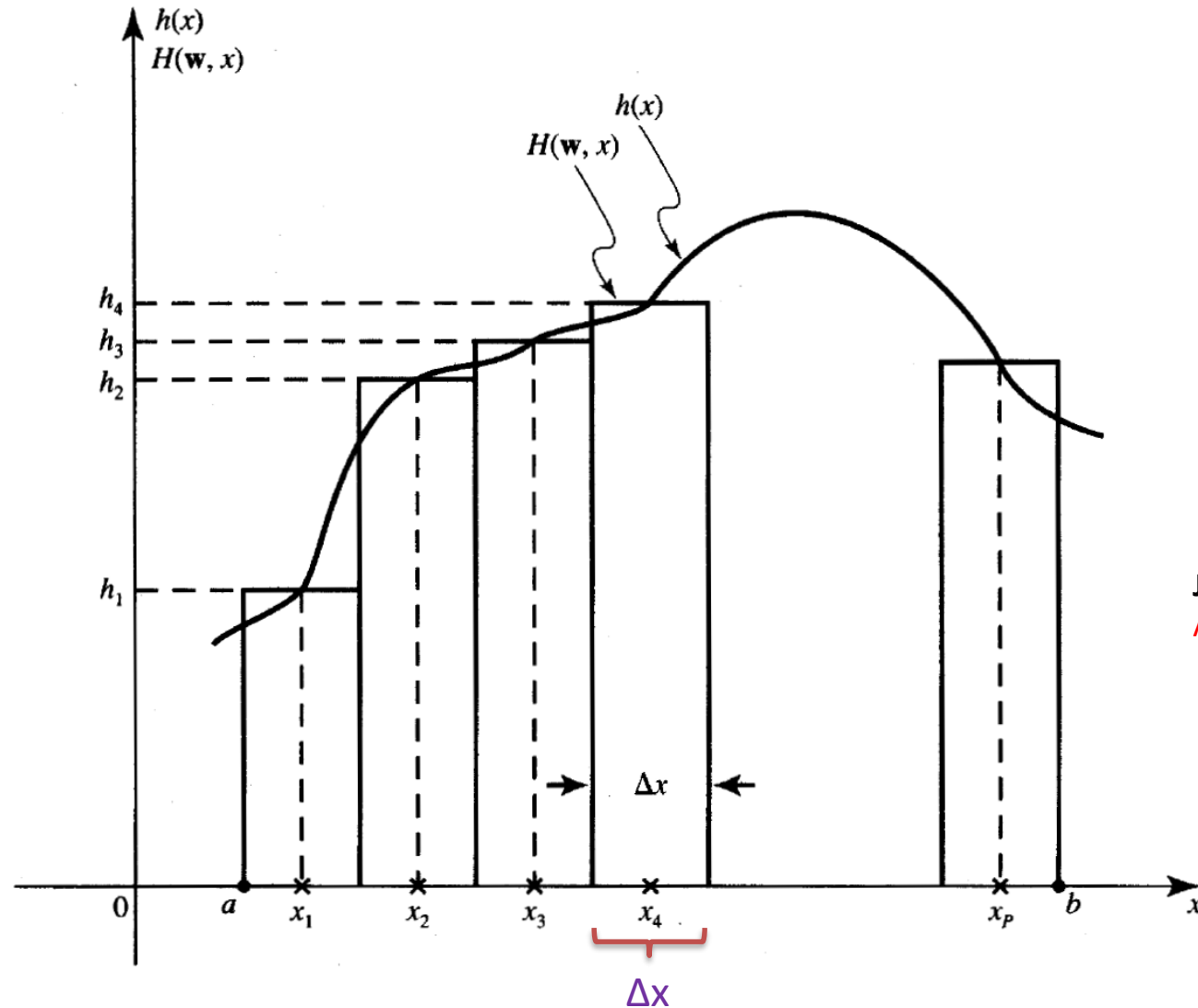
Average error over training set

Find a weight W_* such that :

$$E(W_*) \leq E(W) \text{ for any } W \neq W_*$$

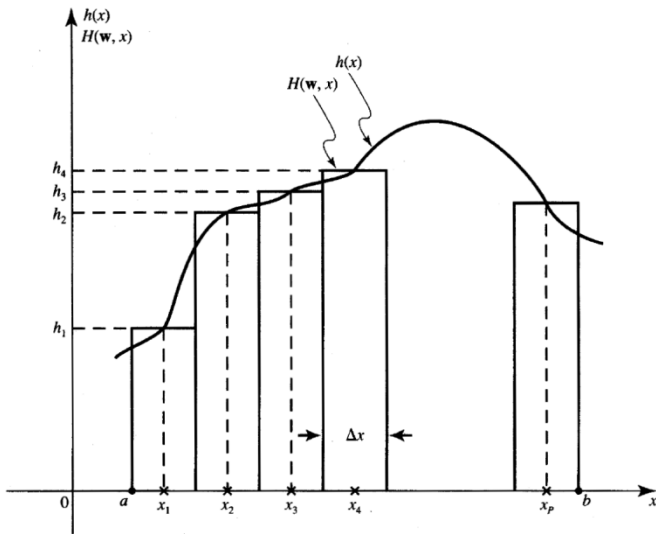
Can we make $E(W_*)$ arbitrarily small ?

- Simplest case : $h : \mathbb{R} \rightarrow \mathbb{R}$
- Approximation interval : $[a,b]$ P samples, sampling interval : $\Delta x = (b-a)/P$



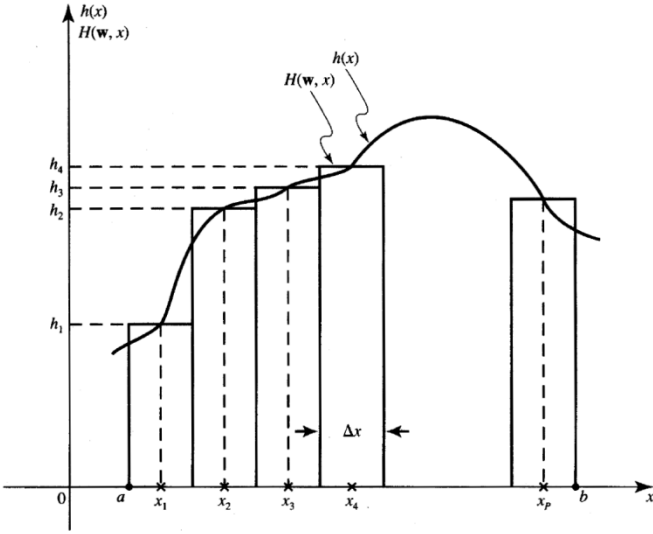
J.M. Zurada,
Artificial Neural Systems

$$\int_a^b h(x) dx \approx \sum_{i=1}^P h(x_i) \Delta x$$



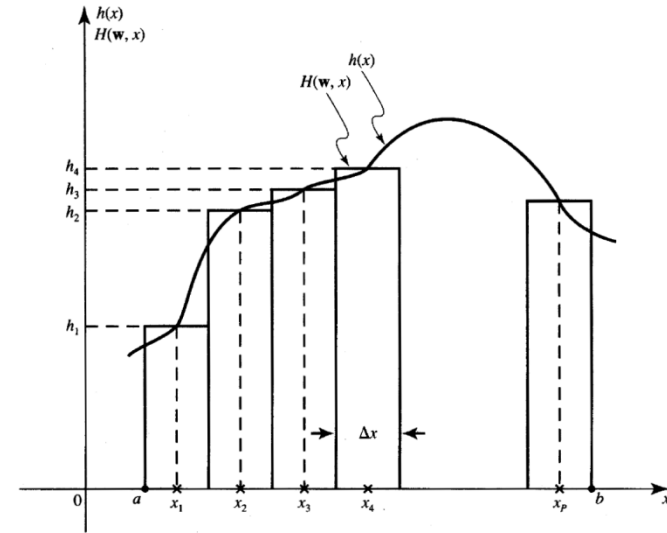
$$\int_a^b h(x)dx \approx \sum_{i=1}^P h(x_i)\Delta x$$

$$h(x) \approx H(W, x)$$



$$\int_a^b h(x) dx \approx \sum_{i=1}^P h(x_i) \Delta x$$

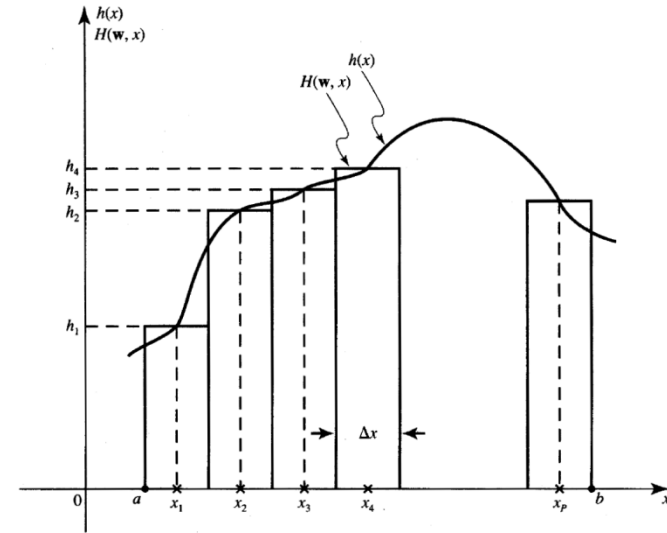
$$h(x) \approx H(W, x)$$



$$\int_a^b | h(x) - H(W, x) | dx \rightarrow 0 \quad as \quad \Delta x \rightarrow 0$$

$$\int_a^b h(x) dx \approx \sum_{i=1}^P h(x_i) \Delta x$$

$$h(x) \approx H(W, x)$$

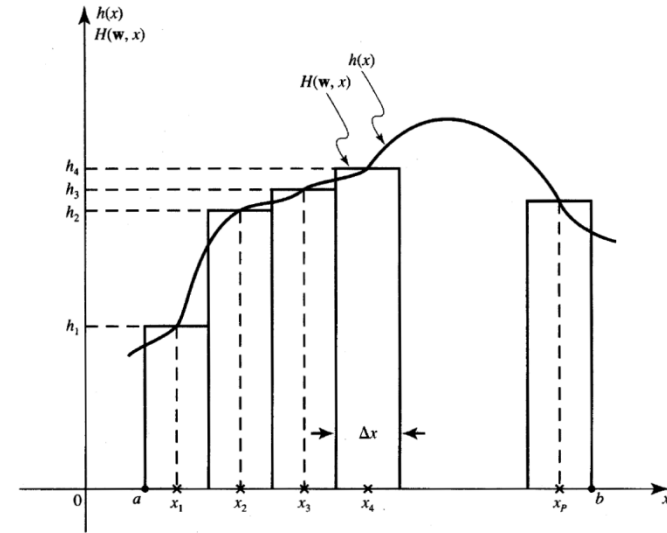


$$\int_a^b | h(x) - H(W, x) | dx \rightarrow 0 \quad as \quad \Delta x \rightarrow 0$$

$$| h(x) - H(W, x) | \quad \text{can be made ARBITRARILY SMALL}$$

$$\int_a^b h(x) dx \approx \sum_{i=1}^P h(x_i) \Delta x$$

$$h(x) \approx H(W, x)$$



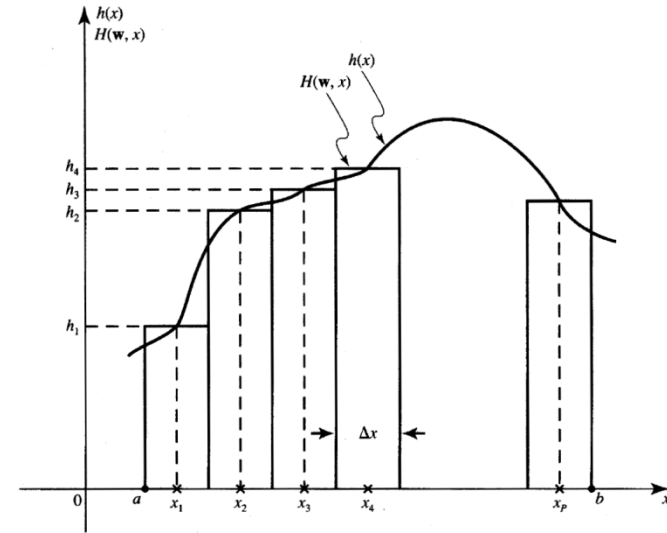
$$\int_a^b | h(x) - H(W, x) | dx \rightarrow 0 \quad as \quad \Delta x \rightarrow 0$$

$$| h(x) - H(W, x) | \quad \text{can be made ARBITRARILY SMALL}$$

- **Conclusion : Staircase Functions are Universal Approximators (Kolmogorov)**

$$\int_a^b h(x) dx \approx \sum_{i=1}^P h(x_i) \Delta x$$

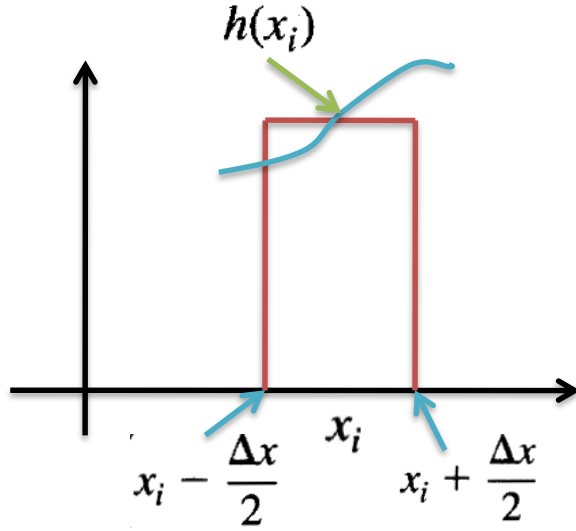
$$h(x) \approx H(W, x)$$



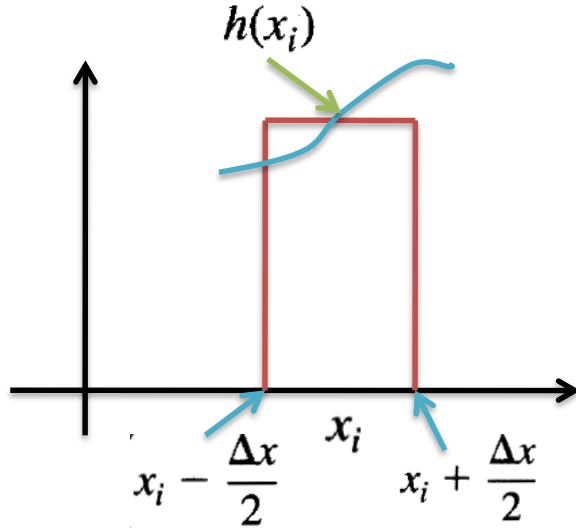
$$\int_a^b | h(x) - H(W, x) | dx \rightarrow 0 \quad as \quad \Delta x \rightarrow 0$$

$| h(x) - H(W, x) |$ can be made **ARBITRARILY SMALL**

- **Conclusion : Staircase Functions are Universal Approximators (Kolmogorov)**
- **PROBLEM :** Can we realize staircase functions by neural networks?

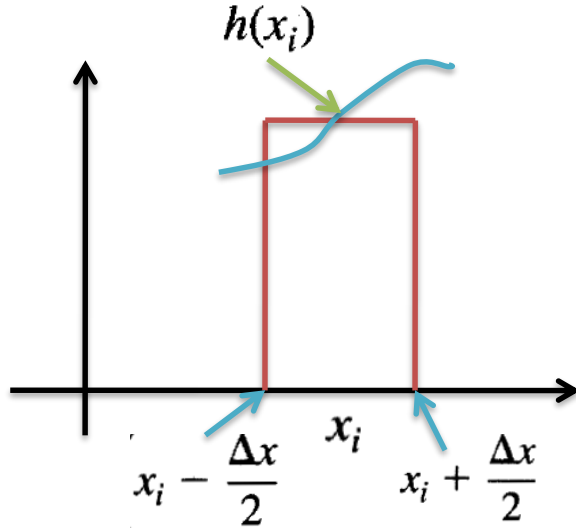


Can we realize this pulse ?



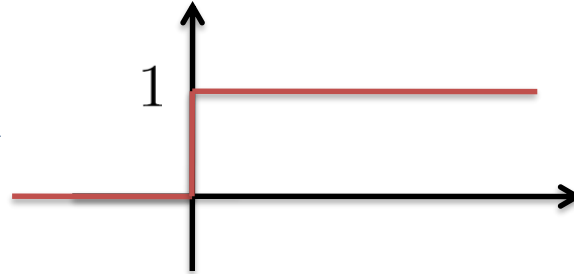
Can we realize this pulse ?

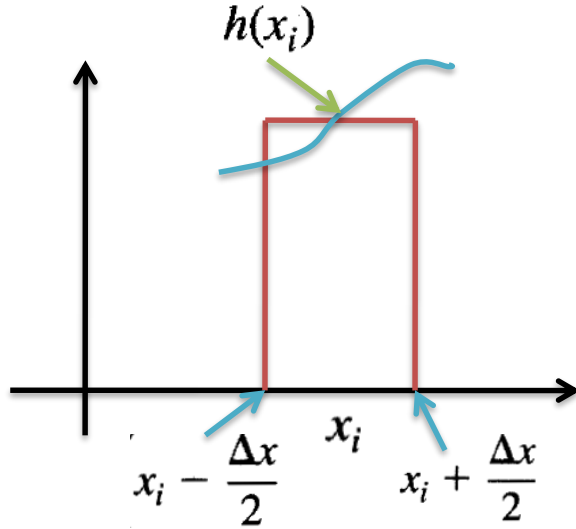
$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



Can we realize this pulse ?

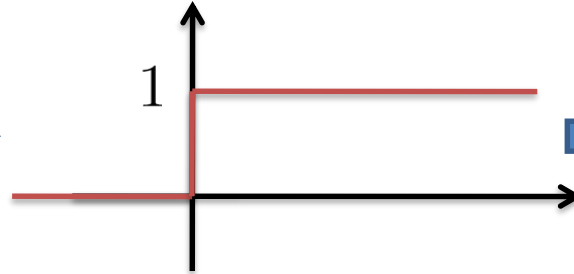
$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



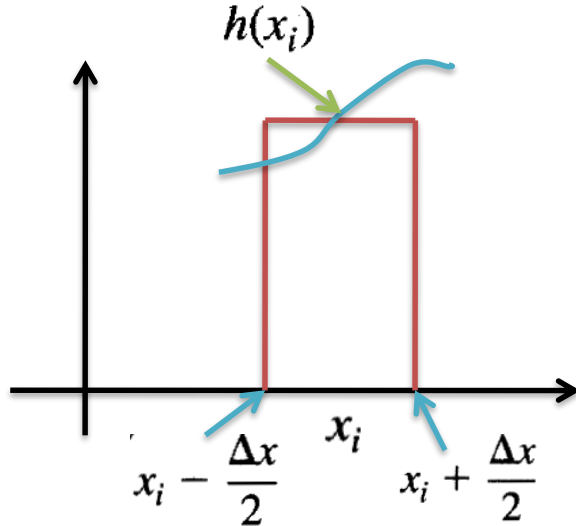


Can we realize this pulse ?

$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

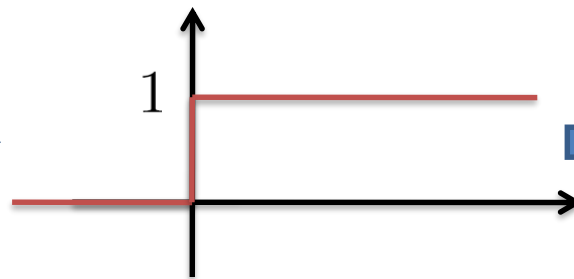


$$1(x) = \frac{1}{2}[\text{sign}(x) + 1]$$



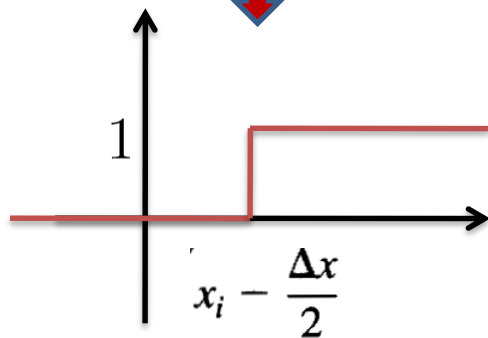
Can we realize this pulse ?

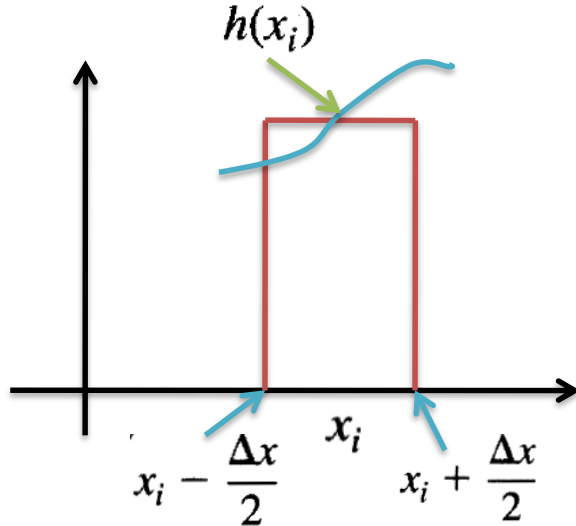
$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



$$1(x) = \frac{1}{2}[\text{sign}(x) + 1]$$

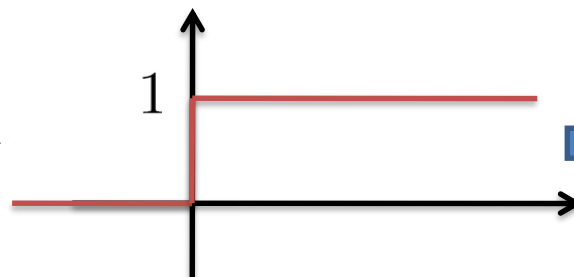
$$1\left(x - \left(x_i - \frac{\Delta x}{2}\right)\right)$$





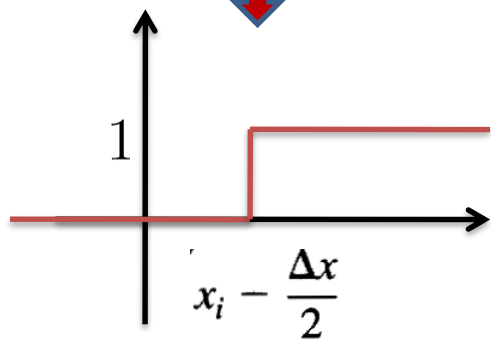
Can we realize this pulse ?

$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

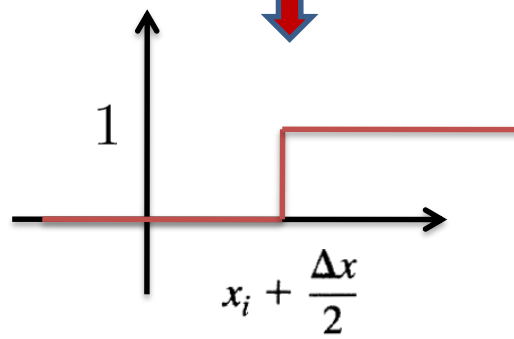


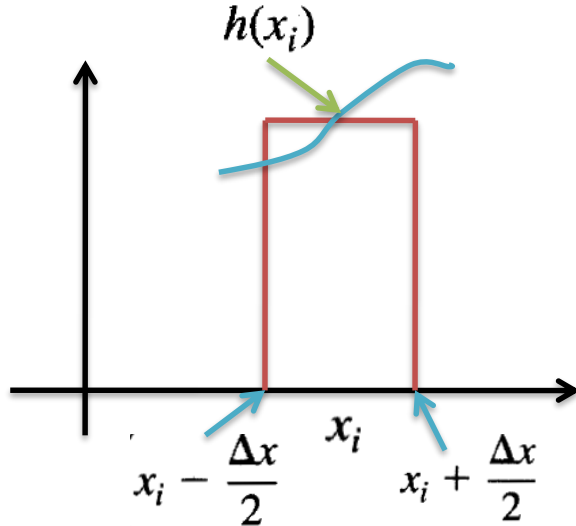
$$1(x) = \frac{1}{2}[\text{sign}(x) + 1]$$

$$1(x - (x_i - \frac{\Delta x}{2}))$$



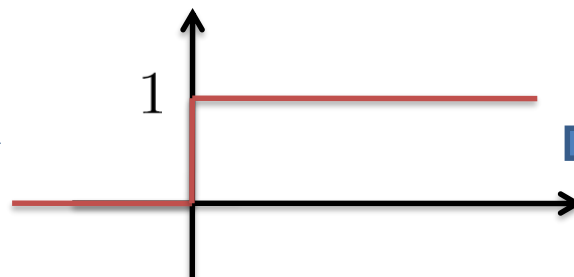
$$1(x - (x_i + \frac{\Delta x}{2}))$$





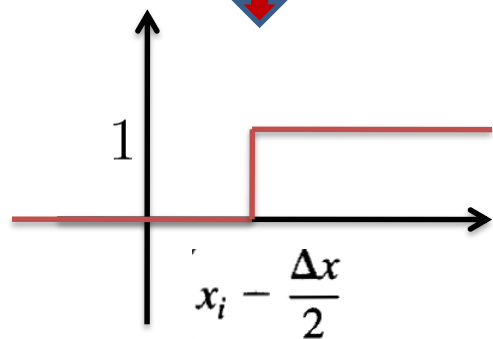
Can we realize this pulse ?

$$1(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

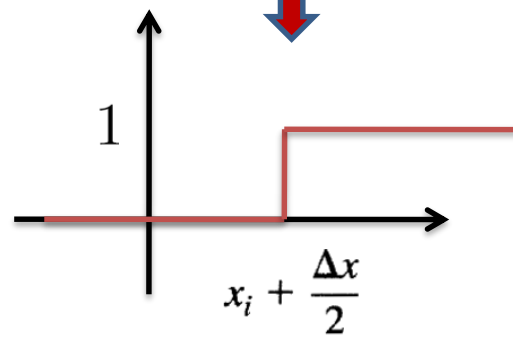


$$1(x) = \frac{1}{2}[\text{sign}(x) + 1]$$

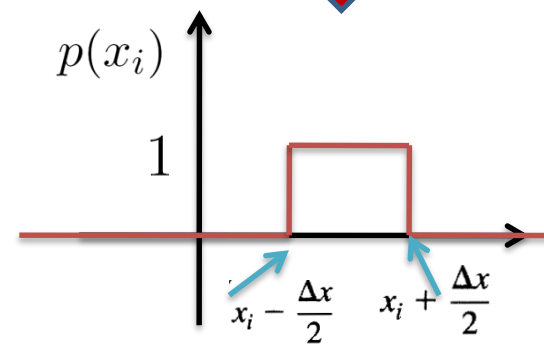
$$1(x - (x_i - \frac{\Delta x}{2}))$$



$$1(x - (x_i + \frac{\Delta x}{2}))$$



$$1(x - (x_i - \frac{\Delta x}{2})) - 1(x - (x_i + \frac{\Delta x}{2}))$$

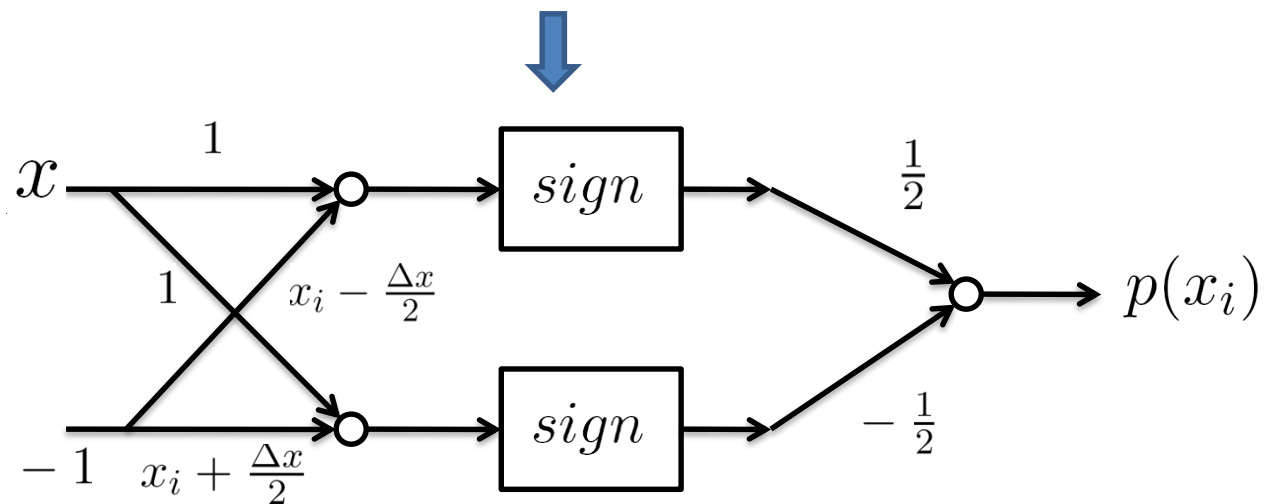


$$\begin{aligned}
 1(x) = \frac{1}{2}[\text{sign}(x) + 1] & \quad \Rightarrow \quad p(x_i) = 1\left(x - \left(x_i - \frac{\Delta x}{2}\right)\right) - 1\left(x - \left(x_i + \frac{\Delta x}{2}\right)\right) \\
 & = \frac{1}{2}\text{sign}\left(x - \left(x_i - \frac{\Delta x}{2}\right)\right) - \frac{1}{2}\text{sign}\left(x - \left(x_i + \frac{\Delta x}{2}\right)\right)
 \end{aligned}$$

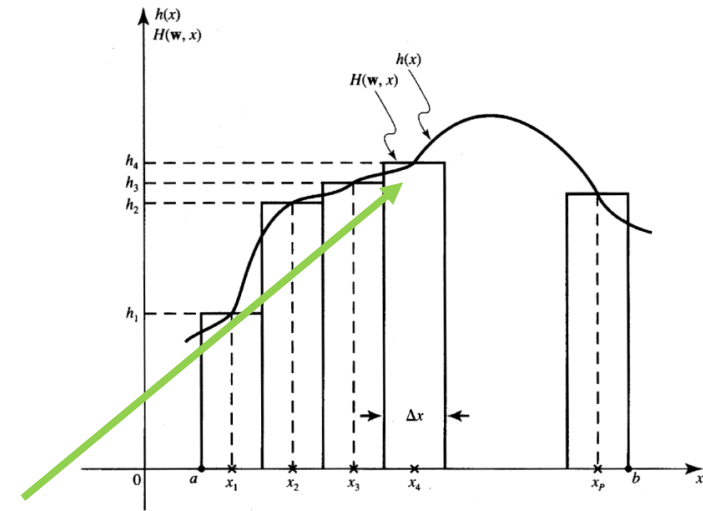
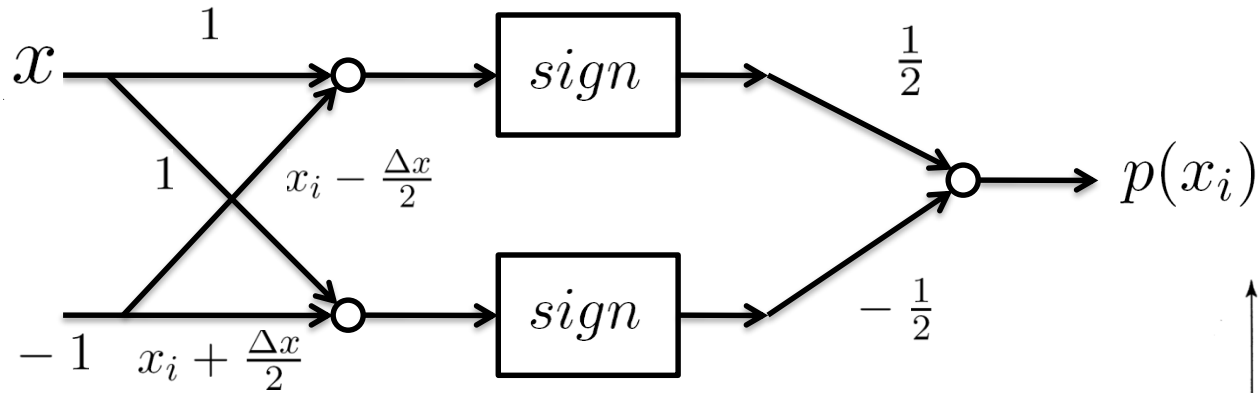
$$1(x) = \frac{1}{2}[\text{sign}(x) + 1] \quad \Rightarrow$$

$$p(x_i) = 1(x - (x_i - \frac{\Delta x}{2})) - 1(x - (x_i + \frac{\Delta x}{2}))$$

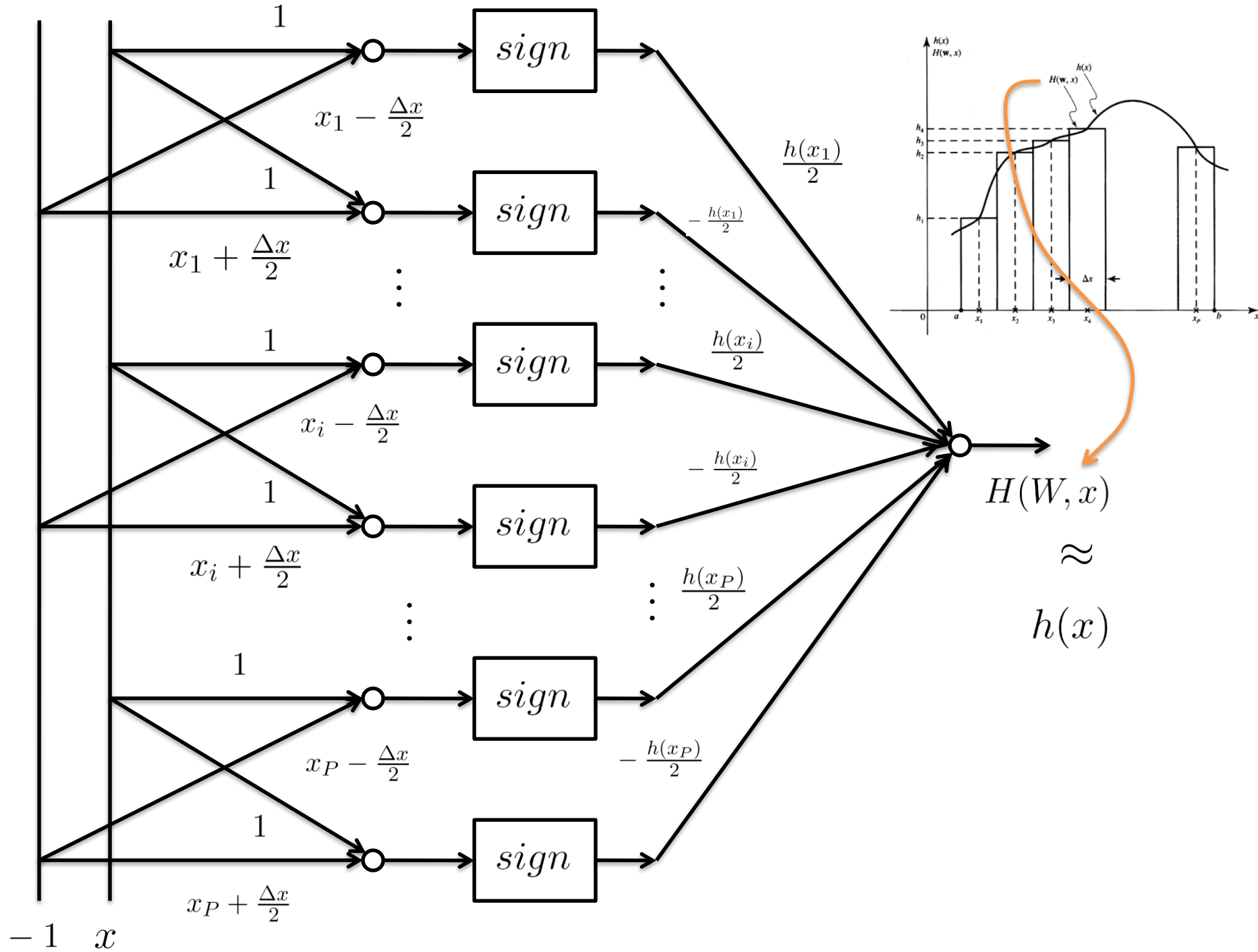
$$= \frac{1}{2}\text{sign}(x - (x_i - \frac{\Delta x}{2})) - \frac{1}{2}\text{sign}(x - (x_i + \frac{\Delta x}{2}))$$



$$\begin{aligned}
 1(x) &= \frac{1}{2}[\text{sign}(x) + 1] \quad \Rightarrow \quad p(x_i) = 1\left(x - \left(x_i - \frac{\Delta x}{2}\right)\right) - 1\left(x - \left(x_i + \frac{\Delta x}{2}\right)\right) \\
 &= \frac{1}{2}\text{sign}\left(x - \left(x_i - \frac{\Delta x}{2}\right)\right) - \frac{1}{2}\text{sign}\left(x - \left(x_i + \frac{\Delta x}{2}\right)\right)
 \end{aligned}$$

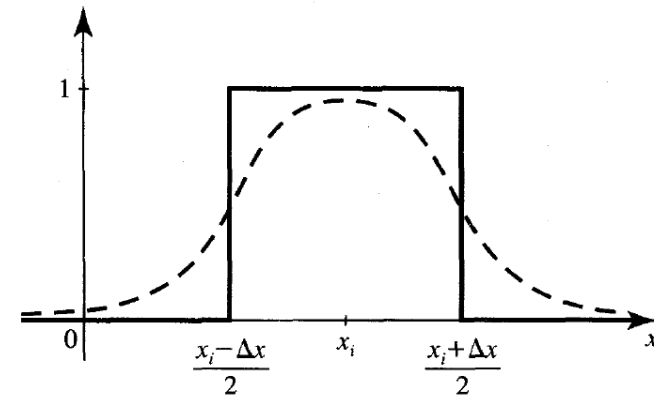
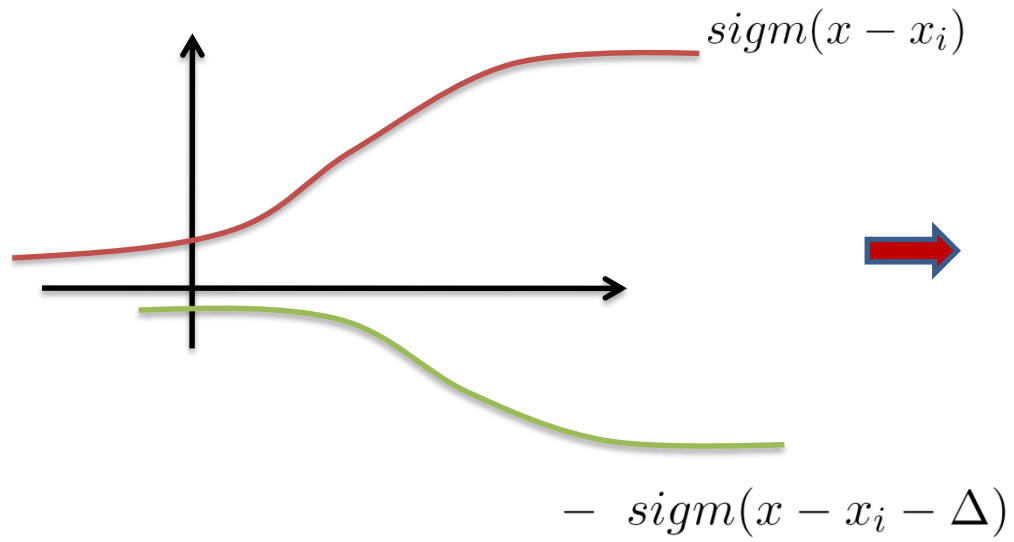


$$H(W, x) = h(x_1)p(x_1) + \dots + h(x_i)p(x_i) + \dots h(x_P)p(x_P)$$

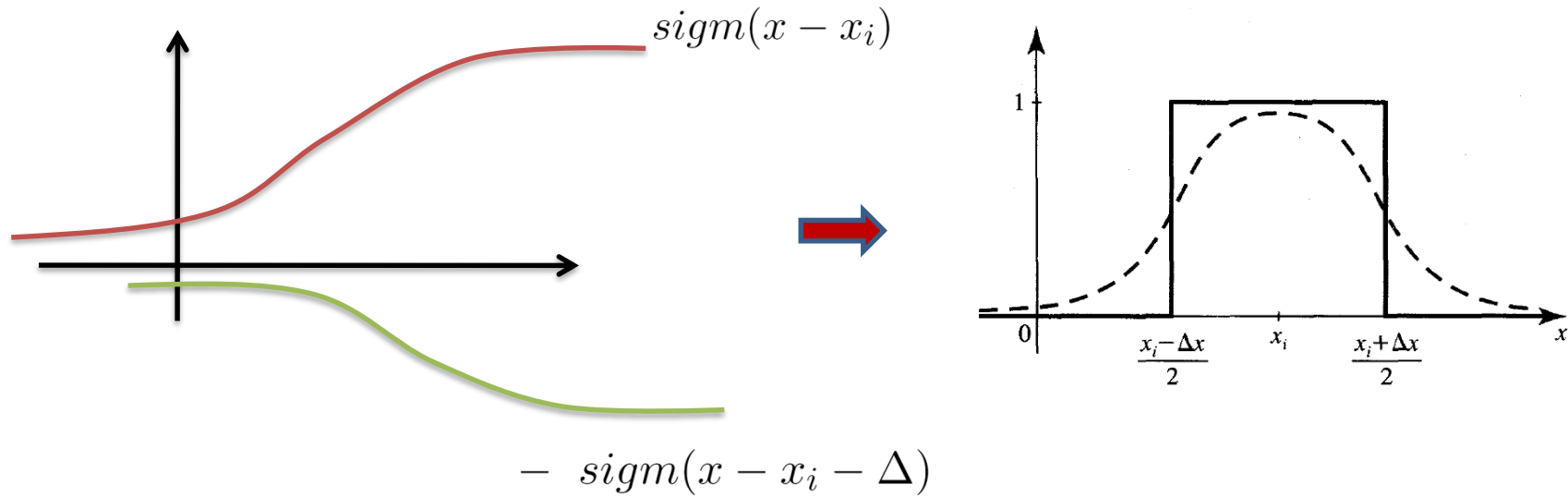


- **Can we approximate PULSE with SIGMOIDS ?**

- Can we approximate PULSE with SIGMOIDS ?

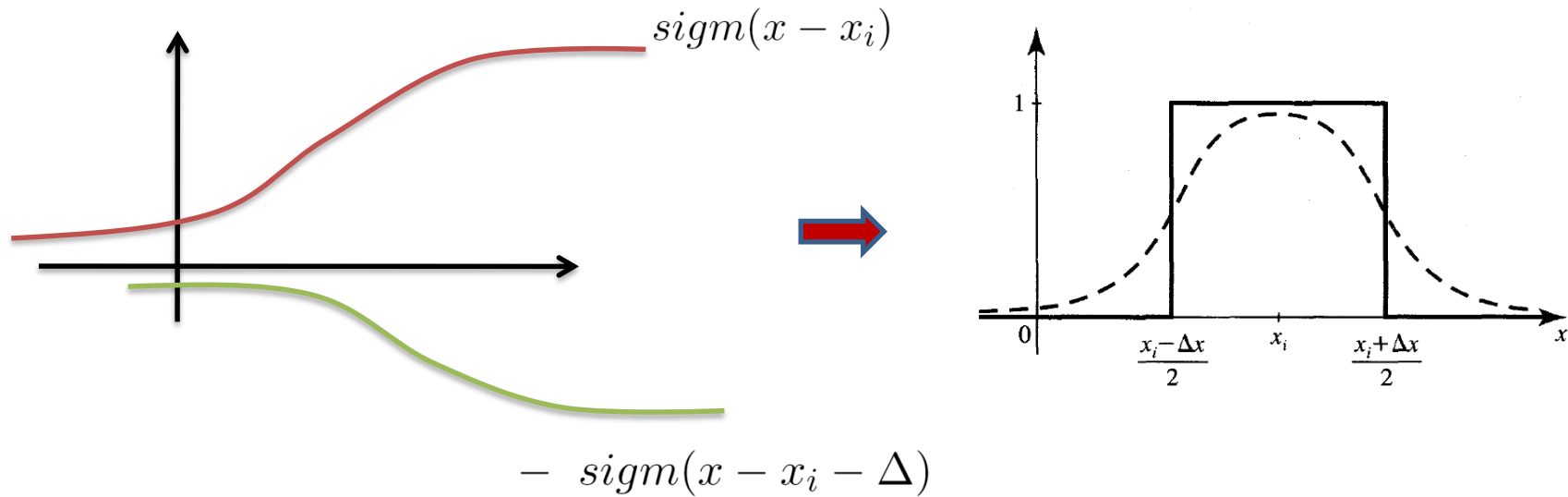


- Can we approximate PULSE with SIGMOIDS ?



- Hence, even if we use SIGMOIDAL FUNCTIONS instead of SIGNUM, we could still approximate the given function $h(x)$

- Can we approximate PULSE with SIGMOIDS ?



- Hence, even if we use SIGMOIDAL FUNCTIONS instead of SIGNUM, we could still approximate the given function $h(x)$
- Same conclusion holds for $h : \mathbf{R}^n \rightarrow \mathbf{R}^m$

- **INITIAL STRUCTURE SELECTION**

- 2 Layer is sufficient for function approximation (may not be optimal)
- 2 layer is also sufficient for many logic functions
- 3 layer may be required for linearly separable patterns...
- Hence initially 2-3 layers may seem to be sufficient...
- How many neurons ?...
- Optimal structure ?...
- Mostly trial/error....
- Still under research....

Given a training set : $\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$
 Given DESIRED OUTPUTS $\{d^1, d^2, \dots, d^N\}$ } **Vectors !**

• Assume that at the **OUTPUT LAYER**, there are **R** neurons. $\Rightarrow d, o \in \mathbf{R}^R$

• Apply n th training pattern $\Rightarrow$ Find $o(n)$

$$\bullet d(n) = \begin{pmatrix} d_1(n) \\ d_2(n) \\ \vdots \\ d_R(n) \end{pmatrix} \quad o(n) = \begin{pmatrix} o_1(n) \\ o_2(n) \\ \vdots \\ o_R(n) \end{pmatrix} \quad n = 1, 2, \dots, N$$

- **Question :** How to measure the error ?

$$e_i = d_i(n) - o_i(n) \quad \Rightarrow \quad \text{Error at } i\text{th output neuron for pattern } n$$

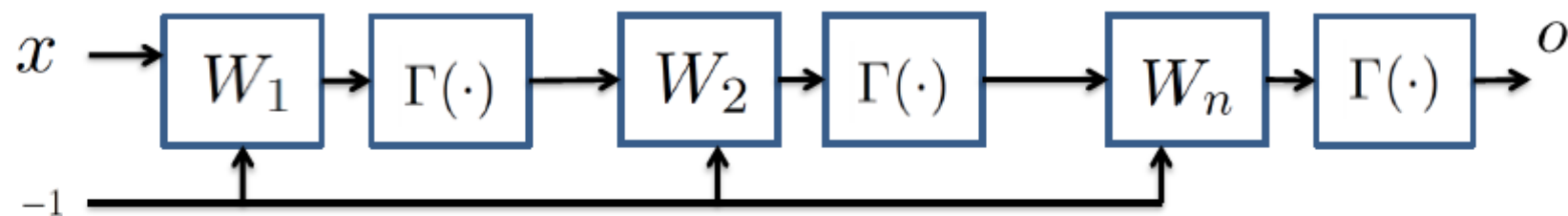
$$\begin{aligned} E(n) &= 0.5 \|d(n) - o(n)\|^2 = 0.5 \sum_{i=1}^R e_i^2(n) \\ &= 0.5 \sum_{i=1}^R (d_i(n) - o_i(n))^2 \end{aligned} \quad \Rightarrow \quad \text{Error for pattern } n$$

$$E_{cum} = \sum_{n=1}^N E(n) = E(1) + E(2) + \dots + E(N) \quad \Rightarrow \quad \text{Cumulative error over training set}$$

$$E_{ave} = \frac{1}{N} E_{cum} \quad \Rightarrow \quad \text{Average error over training set}$$

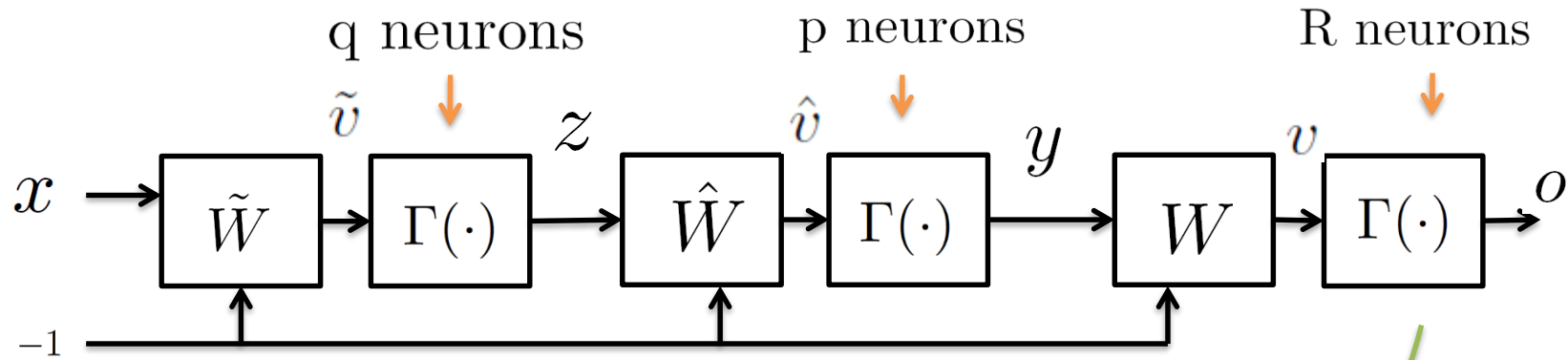
- **ANY WEIGHT WILL BE UPDATED AS FOLLOWS :**

$$W \leftarrow W - \eta \frac{\partial E}{\partial W}$$



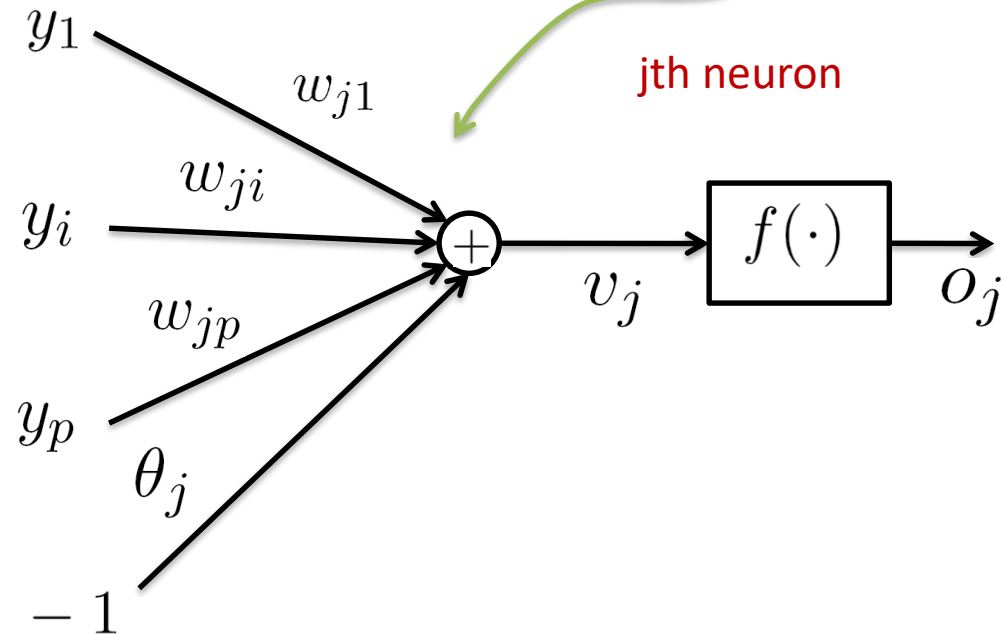
- We don't know what the hidden units ought to do.
- But we can compute how fast the error changes as we change a hidden activity.

- Problem : How to compute error gradient?
- For simplicity, consider 3 layer structure



$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E(n)}{\partial w_{ji}}$$

j: neuron index; 1,2,...,R
i: input index; 1,2,...,p, p+1
n: pattern index, 1,2,...,N



$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial w_{ji}}$?

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial w_{ji}}$?

- Chain of dependence : $E(n) \longrightarrow e_j \longrightarrow o_j \longrightarrow v_j \longrightarrow w_{ji}$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial w_{ji}}$?

- Chain of dependence : $E(n) \longrightarrow e_j \longrightarrow o_j \longrightarrow v_j \longrightarrow w_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial w_{ji}} &= \frac{\partial E(n)}{\partial e_j} \frac{\partial e_j}{\partial o_j} \frac{\partial o_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}} \\ &= e_j(n)(-1)f'(v_j(n))y_i \end{aligned}$$

- $j=1,2,\dots,R$
- $i=1,2,\dots,p,p+1$

for threshold

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial w_{ji}}$?

- Chain of dependence : $E(n) \longrightarrow e_j \longrightarrow o_j \longrightarrow v_j \longrightarrow w_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial w_{ji}} &= \frac{\partial E(n)}{\partial e_j} \frac{\partial e_j}{\partial o_j} \frac{\partial o_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}} \\ &= e_j(n)(-1)f'(v_j(n))y_i \end{aligned}$$

- $j=1,2,\dots,R$
- $i=1,2,\dots,p,p+1$

for threshold

$$w_{ji} \leftarrow w_{ji} + \eta e_j(n) f'(v_j(n)) y_i$$

- $j=1,2,\dots,R$
- $i=1,2,\dots,p,p+1$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial w_{ji}}$?

- Chain of dependence : $E(n) \longrightarrow e_j \longrightarrow o_j \longrightarrow v_j \longrightarrow w_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial w_{ji}} &= \frac{\partial E(n)}{\partial e_j} \frac{\partial e_j}{\partial o_j} \frac{\partial o_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}} \\ &= e_j(n)(-1)f'(v_j(n))y_i \end{aligned}$$

- $j=1,2,\dots,R$
- $i=1,2,\dots,p,p+1$

for threshold

$$f'(v) = \begin{cases} \frac{\lambda}{2} (1 - o^2) & \text{bipolar sigmoidal} \\ \lambda o(1 - o) & \text{unipolar sigmoidal} \end{cases}$$

$$w_{ji} \leftarrow w_{ji} + \eta e_j(n) f'(v_j(n)) y_i$$

- $j=1,2,\dots,R$
- $i=1,2,\dots,p,p+1$

- **Alternative Formulation : LOCAL GRADIENTS**

$$w_{ji} \leftarrow w_{ji} + \eta e_j(n) f'(v_j(n)) y_i$$

- **Alternative Formulation : LOCAL GRADIENTS**

$$w_{ji} \leftarrow w_{ji} + \underbrace{\eta e_j(n) f'(v_j(n))}_{\delta_j(n)} y_i$$

$$\delta_j(n) = e_j(n) f'(v_j(n))$$

local gradient of jth neuron

• **Alternative Formulation : LOCAL GRADIENTS**

$$w_{ji} \leftarrow w_{ji} + \underbrace{\eta e_j(n) f'(v_j(n))}_{\delta_j(n)} y_i$$

$$\delta_j(n) = e_j(n) f'(v_j(n))$$

local gradient of jth neuron



$$w_{ji} \leftarrow w_{ji} + \eta \delta_j(n) y_i \quad \begin{array}{l} \bullet j=1,2,\dots,R \\ \bullet i=1,2,\dots,p,p+1 \end{array}$$

• **Alternative Formulation : LOCAL GRADIENTS**

$$w_{ji} \leftarrow w_{ji} + \underbrace{\eta e_j(n) f'(v_j(n))}_{\delta_j(n)} y_i$$

$$\delta_j(n) = e_j(n) f'(v_j(n))$$

local gradient of jth neuron



$$w_{ji} \leftarrow w_{ji} + \eta \delta_j(n) y_i \quad \begin{array}{l} \bullet j=1,2,\dots,R \\ \bullet i=1,2,\dots,p,p+1 \end{array}$$

• **Matrix Formulation :**

$$\delta(n) = \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_R \end{pmatrix} \quad e(n) = \begin{pmatrix} e_1 \\ \vdots \\ e_R \end{pmatrix} \quad \Gamma'(v) = \begin{pmatrix} f'(v_1) & 0 & \dots & 0 \\ 0 & f'(v_2) & \dots & 0 \\ & & \ddots & \\ 0 & 0 & \dots & f'(v_R) \end{pmatrix}$$



$$\delta(n) = \Gamma'(v) e(n)$$

$$W = \begin{pmatrix} w_{11} & \dots & w_{1p} \\ & \vdots & \\ w_{R1} & \dots & w_{Rp} \end{pmatrix} \quad W_e = [W \mid \theta] \quad y_e = \begin{pmatrix} y \\ -1 \end{pmatrix}$$

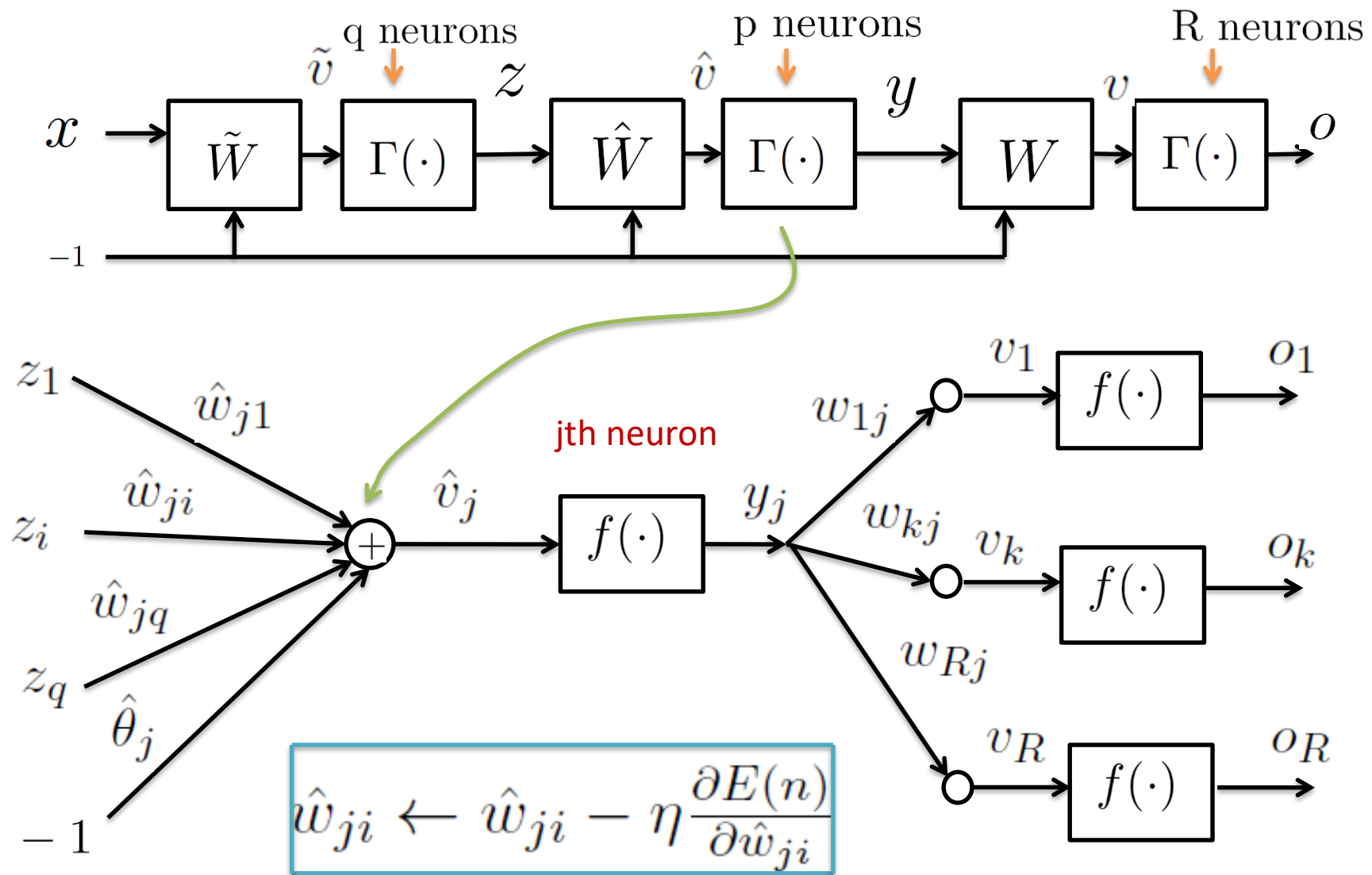
$R \times p$
 $R \times (p+1)$
 $(p+1) \times 1$

$$W_e \leftarrow W_e + \eta \delta(n) y_e^T$$


 $R \times (p+1)$


 $R \times 1$


 $1 \times (p+1)$



j : neuron index; 1,2,...,p
 i : input index; 1,2,...,q, q+1
 n : pattern index, 1,2,...,N

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \hat{w}_{ji}}$?

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \hat{w}_{ji}}$?
- Chain of dependence : $E(n) \rightarrow e_k \rightarrow o_k \rightarrow v_k \rightarrow y_j \rightarrow \hat{v}_j \rightarrow \hat{w}_{ji}$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \hat{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_k \rightarrow o_k \rightarrow v_k \rightarrow y_j \rightarrow \hat{v}_j \rightarrow \hat{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \hat{w}_{ji}} &= \frac{\partial E(n)}{\partial e_k} \frac{\partial e_k}{\partial o_k} \frac{\partial o_k}{\partial v_k} \frac{\partial v_k}{\partial y_j} \frac{\partial y_j}{\partial \hat{v}_j} \frac{\partial \hat{v}_j}{\partial \hat{w}_{ji}} \\ &= \sum_{k=1}^R e_k (-1) f'(v_k) w_{kj} f'(\hat{v}_j) z_i \end{aligned}$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

for threshold

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \hat{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_k \rightarrow o_k \rightarrow v_k \rightarrow y_j \rightarrow \hat{v}_j \rightarrow \hat{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \hat{w}_{ji}} &= \frac{\partial E(n)}{\partial e_k} \frac{\partial e_k}{\partial o_k} \frac{\partial o_k}{\partial v_k} \frac{\partial v_k}{\partial y_j} \frac{\partial y_j}{\partial \hat{v}_j} \frac{\partial \hat{v}_j}{\partial \hat{w}_{ji}} \\ &= \sum_{k=1}^R e_k (-1) f'(v_k) w_{kj} f'(\hat{v}_j) z_i \end{aligned}$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

for threshold

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta f'(\hat{v}_j) \left[\sum_{k=1}^R e_k f'(v_k) w_{kj} \right] z_i$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \hat{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_k \rightarrow o_k \rightarrow v_k \rightarrow y_j \rightarrow \hat{v}_j \rightarrow \hat{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \hat{w}_{ji}} &= \frac{\partial E(n)}{\partial e_k} \frac{\partial e_k}{\partial o_k} \frac{\partial o_k}{\partial v_k} \frac{\partial v_k}{\partial y_j} \frac{\partial y_j}{\partial \hat{v}_j} \frac{\partial \hat{v}_j}{\partial \hat{w}_{ji}} \\ &= \sum_{k=1}^R e_k (-1) f'(v_k) w_{kj} f'(\hat{v}_j) z_i \end{aligned}$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

for threshold

$$f'(v) = \begin{cases} \frac{\lambda}{2} (1 - o^2) & \text{bipolar sigmoidal} \\ \lambda o(1 - o) & \text{unipolar sigmoidal} \end{cases}$$

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta f'(\hat{v}_j) \left[\sum_{k=1}^R e_k f'(v_k) w_{kj} \right] z_i$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

- **Alternative Formulation : LOCAL GRADIENTS**

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta f'(\hat{v}_j) \left[\sum_{k=1}^R e_k f'(v_k) w_{kj} \right] z_i$$

• **Alternative Formulation : LOCAL GRADIENTS**

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta f'(\hat{v}_j) \left[\underbrace{\sum_{k=1}^R e_k f'(v_k) w_{kj}} \right] z_i$$


$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\underbrace{\sum_{k=1}^R e_k f'(v_k) w_{kj}} \right]$$


$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$

local gradient of jth neuron

• **Alternative Formulation : LOCAL GRADIENTS**

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta f'(\hat{v}_j) \left[\underbrace{\sum_{k=1}^R e_k f'(v_k) w_{kj}} \right] z_i$$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\underbrace{\sum_{k=1}^R e_k f'(v_k) w_{kj}} \right]$$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$

local gradient of jth neuron

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta \hat{\delta}_j(n) z_i$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

• **Alternative Formulation : LOCAL GRADIENTS**

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta \underbrace{f'(\hat{v}_j) \left[\sum_{k=1}^R e_k f'(v_k) w_{kj} \right]}_{\delta_j(n)} z_i$$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R e_k \underbrace{f'(v_k) w_{kj}}_{\delta_k(n)} \right]$$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$

local gradient of jth neuron

$$\hat{w}_{ji} \leftarrow \hat{w}_{ji} + \eta \hat{\delta}_j(n) z_i$$

- $j=1,2,\dots,p$
- $i=1,2,\dots,q,q+1$

• **Matrix Formulation :**

$$\hat{\delta}(n) = \begin{pmatrix} \hat{\delta}_1 \\ \vdots \\ \hat{\delta}_p \end{pmatrix} \quad \Gamma'(\hat{v}) = \begin{pmatrix} f'(\hat{v}_1) & 0 & \dots & 0 \\ 0 & f'(\hat{v}_2) & \dots & 0 \\ & & \vdots & \\ 0 & \dots & & f'(\hat{v}_p) \end{pmatrix}$$

$$\hat{W} = \begin{pmatrix} \hat{w}_{11} & \dots & \hat{w}_{1q} \\ & \vdots & \\ \hat{w}_{p1} & \dots & \hat{w}_{pq} \end{pmatrix}$$

p x q

$$\hat{W}_e = [\hat{W} \mid \hat{\theta}]$$

p x (q+1)

$$z_e = \begin{pmatrix} z \\ -1 \end{pmatrix}$$

(q+1) x 1

$$\hat{W} = \begin{pmatrix} \hat{w}_{11} & \dots & \hat{w}_{1q} \\ & \vdots & \\ \hat{w}_{p1} & \dots & \hat{w}_{pq} \end{pmatrix} \quad \hat{W}_e = \begin{bmatrix} \hat{W} & | & \hat{\theta} \end{bmatrix} \quad z_e = \begin{pmatrix} z \\ -1 \end{pmatrix}$$

$p \times q$
 $p \times (q+1)$
 $(q+1) \times 1$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$

$$\hat{W} = \begin{pmatrix} \hat{w}_{11} & \dots & \hat{w}_{1q} \\ & \vdots & \\ \hat{w}_{p1} & \dots & \hat{w}_{pq} \end{pmatrix} \quad \hat{W}_e = \begin{bmatrix} \hat{W} & | & \hat{\theta} \end{bmatrix} \quad z_e = \begin{pmatrix} z \\ -1 \end{pmatrix}$$

$p \times q$
 $p \times (q+1)$
 $(q+1) \times 1$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$



$$\hat{\delta}(n) = \Gamma'(\hat{v}) W^T \delta(n)$$

$$\hat{W} = \begin{pmatrix} \hat{w}_{11} & \dots & \hat{w}_{1q} \\ & \vdots & \\ \hat{w}_{p1} & \dots & \hat{w}_{pq} \end{pmatrix} \quad \hat{W}_e = \begin{bmatrix} \hat{W} & | & \hat{\theta} \end{bmatrix} \quad z_e = \begin{pmatrix} z \\ -1 \end{pmatrix}$$

$p \times q$
 $p \times (q+1)$
 $(q+1) \times 1$

$$\hat{\delta}_j(n) = f'(\hat{v}_j) \left[\sum_{k=1}^R \delta_k(n) w_{kj} \right]$$



$$\hat{\delta}(n) = \Gamma'(\hat{v}) W^T \delta(n)$$

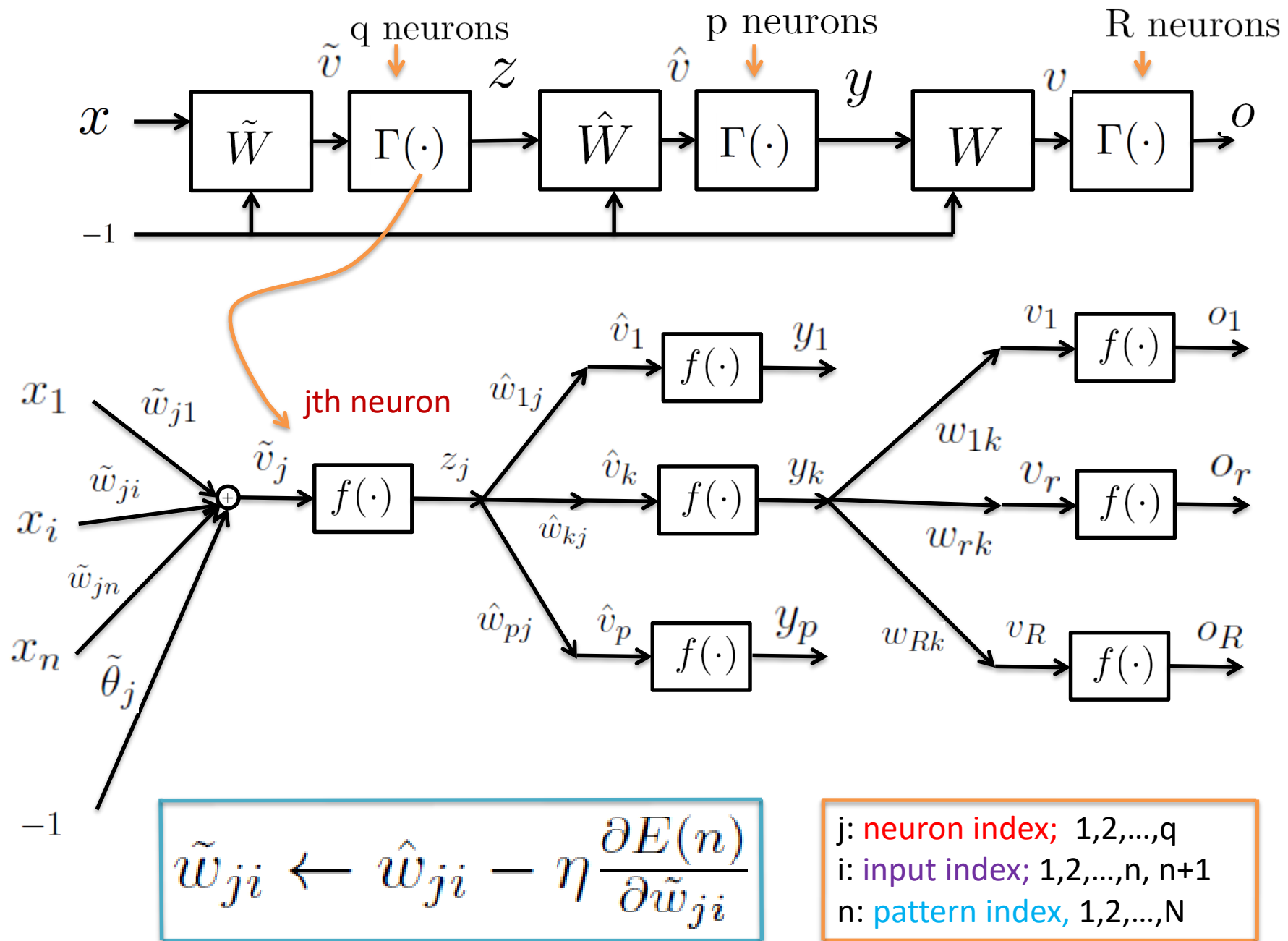


$$\hat{W}_e \leftarrow \hat{W}_e + \eta \hat{\delta}(n) z_e^T$$

$p \times (q+1)$

$p \times 1$

$1 \times (q+1)$



$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

• How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

• Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \tilde{w}_{ji}} &= \sum_{k=1}^p \sum_{r=1}^R \frac{\partial E(n)}{\partial e_r} \frac{\partial e_r}{\partial o_r} \frac{\partial o_r}{\partial v_r} \frac{\partial v_r}{\partial y_k} \frac{\partial y_k}{\partial \hat{v}_k} \frac{\partial \hat{v}_k}{\partial z_j} \frac{\partial z_j}{\partial \tilde{v}_j} \frac{\partial \tilde{v}_j}{\partial \tilde{w}_{ji}} \\ &= \sum_{k=1}^p \sum_{r=1}^R e_r (-1) f'(v_r) w_{rk} f'(\hat{v}_k) \hat{w}_{kj} f'(\tilde{v}_j) x_i \end{aligned}$$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \tilde{w}_{ji}} &= \sum_{k=1}^p \sum_{r=1}^R \frac{\partial E(n)}{\partial e_r} \frac{\partial e_r}{\partial o_r} \frac{\partial o_r}{\partial v_r} \frac{\partial v_r}{\partial y_k} \frac{\partial y_k}{\partial \hat{v}_k} \frac{\partial \hat{v}_k}{\partial z_j} \frac{\partial z_j}{\partial \tilde{v}_j} \frac{\partial \tilde{v}_j}{\partial \tilde{w}_{ji}} \\ &= \sum_{k=1}^p \sum_{r=1}^R e_r (-1) f'(v_r) w_{rk} f'(\hat{v}_k) \hat{w}_{kj} f'(\tilde{v}_j) x_i \\ &= -f'(\tilde{v}_j) [\sum_{k=1}^p f'(\hat{v}_k) \sum_{r=1}^R (e_r f'(v_r) w_{rk}) \hat{w}_{kj}] x_i \end{aligned}$$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

- How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

- Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \tilde{w}_{ji}} &= \sum_{k=1}^p \sum_{r=1}^R \frac{\partial E(n)}{\partial e_r} \frac{\partial e_r}{\partial o_r} \frac{\partial o_r}{\partial v_r} \frac{\partial v_r}{\partial y_k} \frac{\partial y_k}{\partial \hat{v}_k} \frac{\partial \hat{v}_k}{\partial z_j} \frac{\partial z_j}{\partial \tilde{v}_j} \frac{\partial \tilde{v}_j}{\partial \tilde{w}_{ji}} \\ &= \sum_{k=1}^p \sum_{r=1}^R e_r (-1) f'(v_r) w_{rk} f'(\hat{v}_k) \hat{w}_{kj} f'(\tilde{v}_j) x_i \\ &= -f'(\tilde{v}_j) \left[\sum_{k=1}^p f'(\hat{v}_k) \underbrace{\sum_{r=1}^R (e_r f'(v_r) w_{rk})}_{\delta_r} \hat{w}_{kj} \right] x_i \end{aligned}$$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

• How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

• Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$\begin{aligned} \frac{\partial E(n)}{\partial \tilde{w}_{ji}} &= \sum_{k=1}^p \sum_{r=1}^R \frac{\partial E(n)}{\partial e_r} \frac{\partial e_r}{\partial o_r} \frac{\partial o_r}{\partial v_r} \frac{\partial v_r}{\partial y_k} \frac{\partial y_k}{\partial \hat{v}_k} \frac{\partial \hat{v}_k}{\partial z_j} \frac{\partial z_j}{\partial \tilde{v}_j} \frac{\partial \tilde{v}_j}{\partial \tilde{w}_{ji}} \\ &= \sum_{k=1}^p \sum_{r=1}^R e_r (-1) f'(v_r) w_{rk} f'(\hat{v}_k) \hat{w}_{kj} f'(\tilde{v}_j) x_i \\ &= -f'(\tilde{v}_j) \left[\underbrace{\sum_{k=1}^p f'(\hat{v}_k) \underbrace{\sum_{r=1}^R (e_r f'(v_r) w_{rk})}_{\delta_r}}_{\hat{\delta}_k} \right] \hat{w}_{kj} x_i \end{aligned}$$

$$E(n) = 0.5 \sum_{i=1}^R e_i^2(n) = 0.5(e_1^2(n) + \dots + e_j^2(n) + \dots + e_R^2(n))$$

• How to compute $\frac{\partial E(n)}{\partial \tilde{w}_{ji}}$?

• Chain of dependence : $E(n) \rightarrow e_r \rightarrow o_r \rightarrow v_r \rightarrow y_k \rightarrow \hat{v}_k \rightarrow z_j \rightarrow \tilde{v}_j \rightarrow \tilde{w}_{ji}$

$$\begin{aligned}
 \frac{\partial E(n)}{\partial \tilde{w}_{ji}} &= \sum_{k=1}^p \sum_{r=1}^R \frac{\partial E(n)}{\partial e_r} \frac{\partial e_r}{\partial o_r} \frac{\partial o_r}{\partial v_r} \frac{\partial v_r}{\partial y_k} \frac{\partial y_k}{\partial \hat{v}_k} \frac{\partial \hat{v}_k}{\partial z_j} \frac{\partial z_j}{\partial \tilde{v}_j} \frac{\partial \tilde{v}_j}{\partial \tilde{w}_{ji}} \\
 &= \sum_{k=1}^p \sum_{r=1}^R e_r (-1) f'(v_r) w_{rk} f'(\hat{v}_k) \hat{w}_{kj} f'(\tilde{v}_j) x_i \\
 &= -f'(\tilde{v}_j) \left[\sum_{k=1}^p f'(\hat{v}_k) \underbrace{\sum_{r=1}^R (e_r f'(v_r) w_{rk})}_{\delta_r} \hat{w}_{kj} \right] x_i \\
 &= -f'(\tilde{v}_j) \underbrace{\left[\sum_{k=1}^p \hat{\delta}_k \hat{w}_{kj} \right]}_{\tilde{\delta}_j} x_i = -\tilde{\delta}_j x_i
 \end{aligned}$$

• **Matrix Formulation :**

$$\tilde{\delta}(n) = \begin{pmatrix} \hat{\delta}_1 \\ \vdots \\ \hat{\delta}_q \end{pmatrix} \quad \Gamma'(\tilde{v}) = \begin{pmatrix} f'(\tilde{v}_1) & 0 & \dots & 0 \\ 0 & f'(\tilde{v}_2) & \dots & 0 \\ & & \vdots & \\ 0 & \dots & & f'(\tilde{v}_q) \end{pmatrix}$$

• **Matrix Formulation :**

$$\tilde{\delta}(n) = \begin{pmatrix} \hat{\delta}_1 \\ \vdots \\ \hat{\delta}_q \end{pmatrix} \quad \Gamma'(\tilde{v}) = \begin{pmatrix} f'(\tilde{v}_1) & 0 & \dots & 0 \\ 0 & f'(\tilde{v}_2) & \dots & 0 \\ & & \vdots & \\ 0 & \dots & & f'(\tilde{v}_q) \end{pmatrix}$$

$$\tilde{W} = \begin{pmatrix} \tilde{w}_{11} & \dots & \tilde{w}_{1n} \\ & \vdots & \\ \tilde{w}_{q1} & \dots & \tilde{w}_{qn} \end{pmatrix} \quad \tilde{W}_e = [\tilde{W} \mid \tilde{\theta}] \quad x_e = \begin{pmatrix} x \\ -1 \end{pmatrix}$$

q x n

q x (n+1) (n+1) x 1

• **Matrix Formulation :**

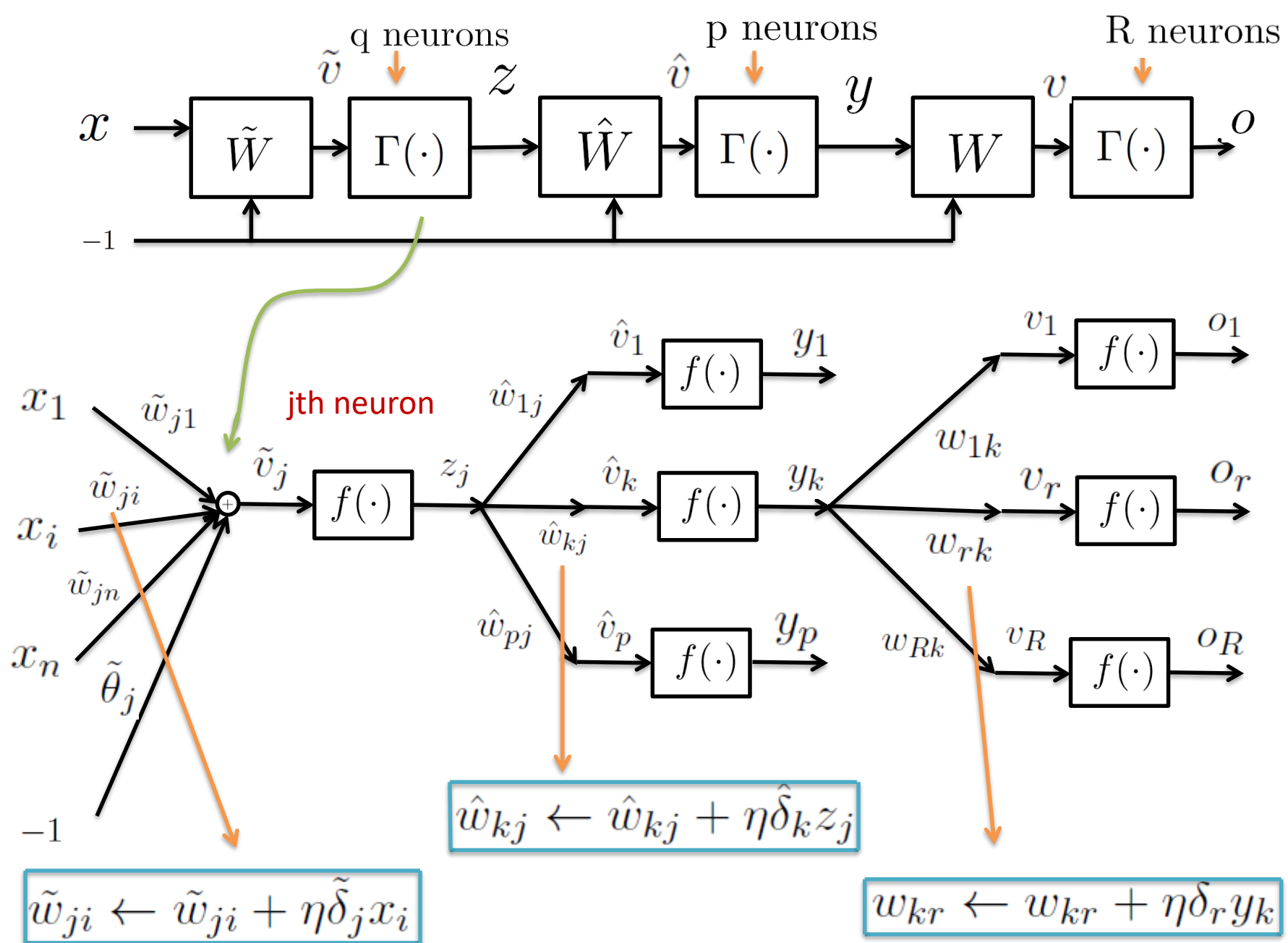
$$\tilde{\delta}(n) = \begin{pmatrix} \hat{\delta}_1 \\ \vdots \\ \hat{\delta}_q \end{pmatrix} \quad \Gamma'(\tilde{v}) = \begin{pmatrix} f'(\tilde{v}_1) & 0 & \dots & 0 \\ 0 & f'(\tilde{v}_2) & \dots & 0 \\ & & \vdots & \\ 0 & \dots & & f'(\tilde{v}_q) \end{pmatrix}$$

$$\tilde{W} = \begin{pmatrix} \tilde{w}_{11} & \dots & \tilde{w}_{1n} \\ & \vdots & \\ \tilde{w}_{q1} & \dots & \tilde{w}_{qn} \end{pmatrix} \quad \tilde{W}_e = [\tilde{W} \mid \tilde{\theta}] \quad x_e = \begin{pmatrix} x \\ -1 \end{pmatrix}$$

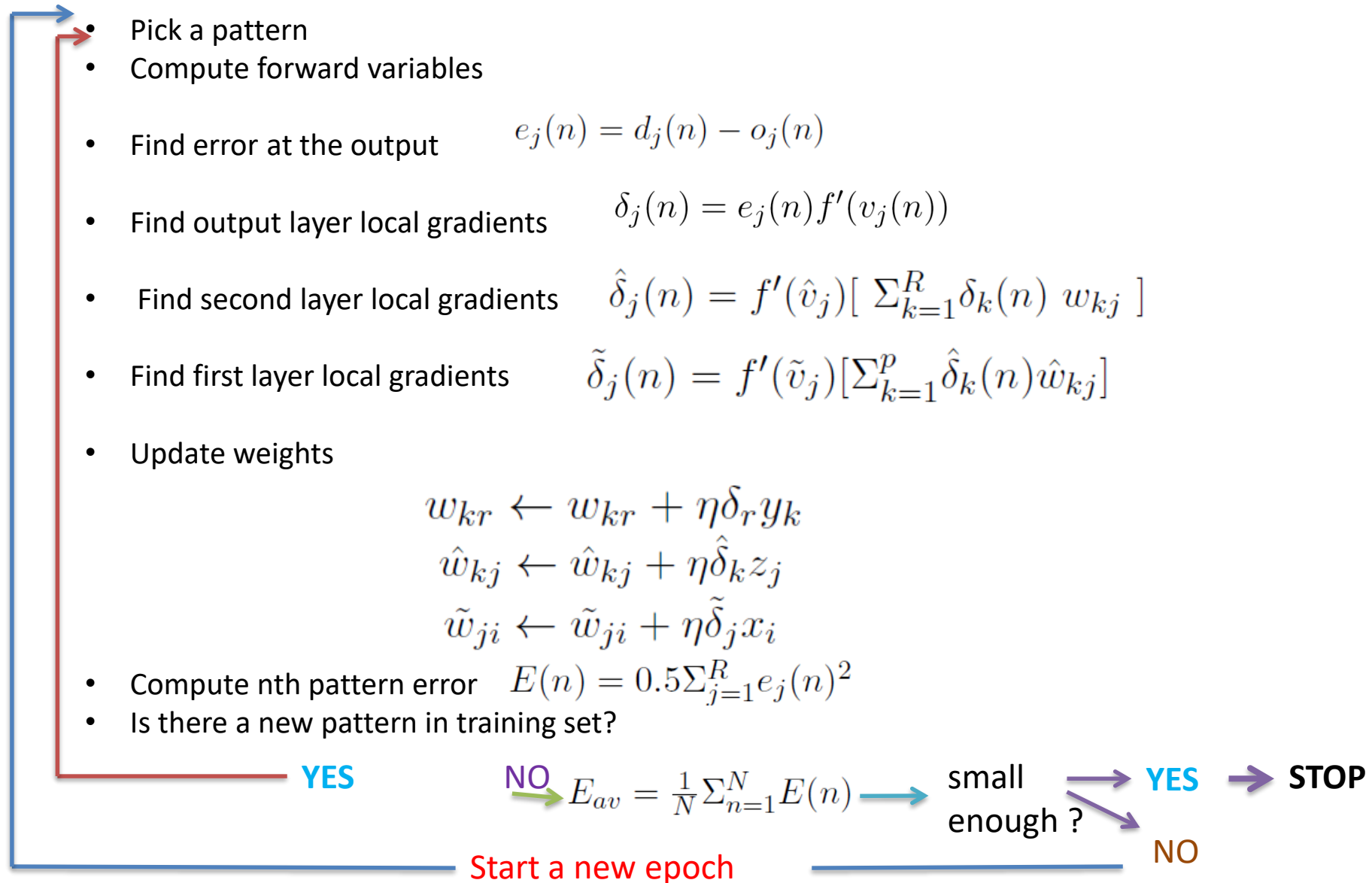
q x n q x (n+1) (n+1) x 1

$$\tilde{\delta}(n) = \Gamma'(\tilde{v}) \tilde{W}^T \hat{\delta}(n)$$

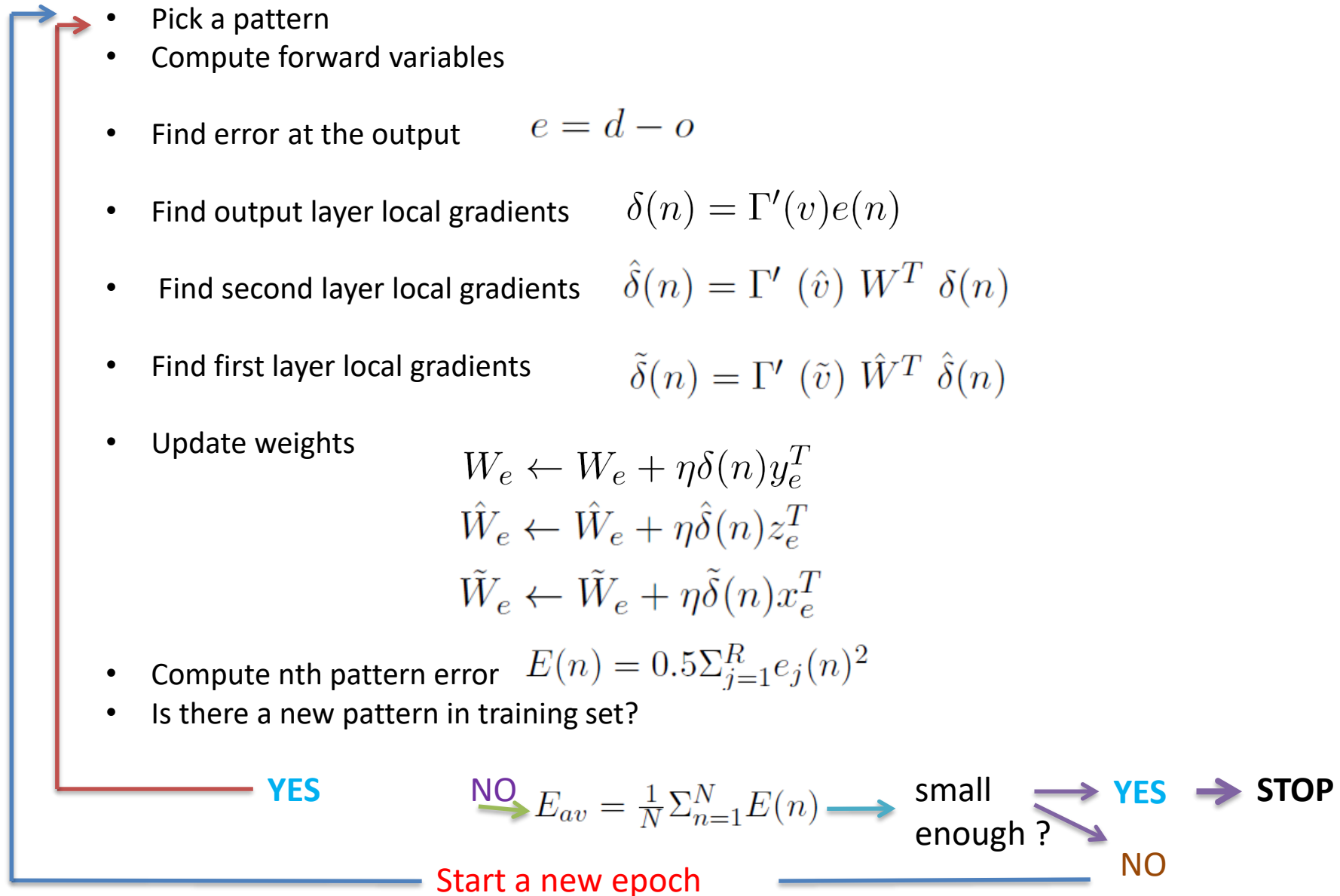
$$\tilde{W}_e \leftarrow \underset{\substack{\downarrow \\ \text{q x (n+1)}}}{\tilde{W}_e} + \underset{\substack{\downarrow \\ \text{q x 1}}}{\eta \tilde{\delta}(n)} \underset{\substack{\searrow \\ \text{1 x (n+1)}}}{x_e^T}$$



- 3 layer backpropagation - COMPONENTWISE



- 3 layer backpropagation -MATRIXWISE





Epoch
000,427

Learning rate

0.03

Activation

ReLU

Regularization

None

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 2 HIDDEN LAYERS

+ -

4 neurons

+ -

2 neurons

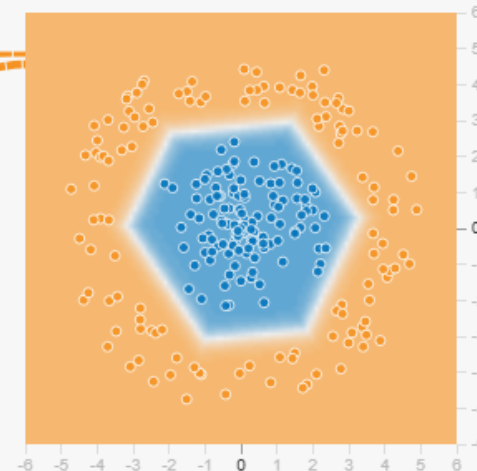
The outputs are mixed with varying **weights**, shown by the thickness of the lines.

This is the output from one **neuron**. Hover to see it larger.

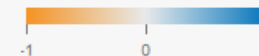
OUTPUT

Test loss 0.001

Training loss 0.001



Colors shows data, neuron and weight values.



<http://playground.tensorflow.org/>

Limits of End-to-End Learning

Tobias Glasmachers

TOBIAS.GLASMACHERS@INI.RUB.DE

Institute for Neural Computation, Ruhr-University Bochum, Germany

Editors: Yung-Kyun Noh and Min-Ling Zhang

Abstract

End-to-end learning refers to training a possibly complex learning system by applying gradient-based learning to the system as a whole. End-to-end learning systems are specifically designed so that all modules are differentiable. In effect, not only a central learning machine, but also all “peripheral” modules like representation learning and memory formation are covered by a holistic learning process. The power of end-to-end learning has been demonstrated on many tasks, like playing a whole array of Atari video games with a single architecture. While pushing for solutions to more challenging tasks, network architectures keep growing more and more complex.

In this paper we ask the question whether and to what extent end-to-end learning is a future-proof technique in the sense of *scaling* to complex and diverse data processing architectures. We point out potential inefficiencies, and we argue in particular that end-to-end learning does not make optimal use of the modular design of present neural networks. Our surprisingly simple experiments demonstrate these inefficiencies, up to the complete breakdown of learning.

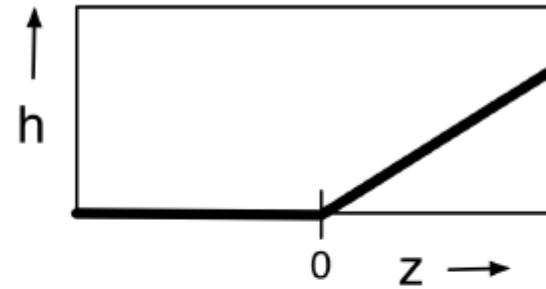
Keywords: end-to-end machine learning

Question 1: Backprop Example

Now let's work through a backprop example with numbers.

The hidden layer consists of rectified linear units. As a refresher,

$$h_i = \max(z_i, 0)$$
$$\frac{dh_i}{dz_i} = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i \leq 0. \end{cases}$$



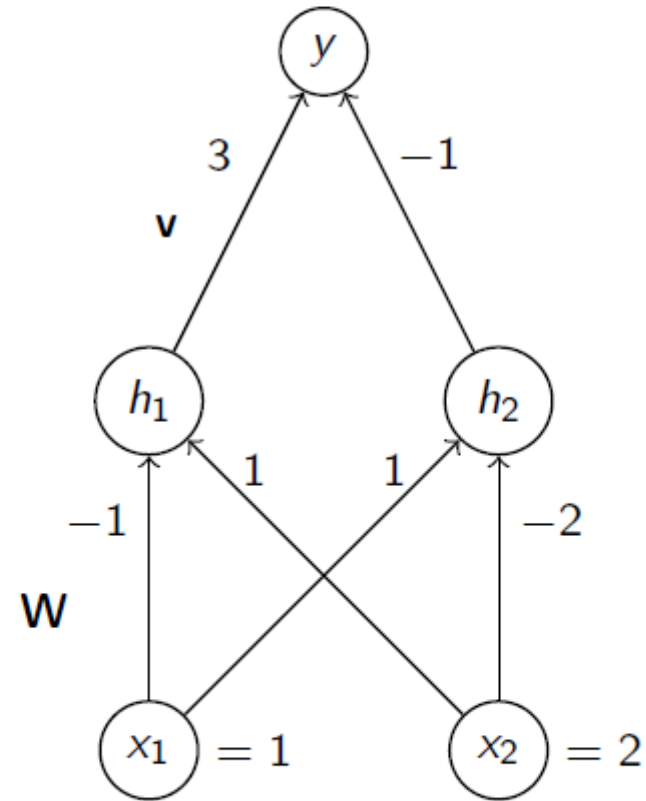
Question 1: Backprop Example

Data : $x = (1, 2)$, $t = 1$.

Parameters : $\mathbf{W}$: 2×2 matrix, $\mathbf{v}$: 2-dimensional vector.

Each hidden unit has the rectified linear non-linearity. $h = \max(0, z)$

Output unit is linear. Loss function: $C = \frac{1}{2}(y - t)^2$.



Question 1: Backprop Example

Data : $x = (1, 2)$, $t = 1$.

Parameters : $\mathbf{W}$: 2×2 matrix, $\mathbf{v}$: 2-dimensional vector.

Each hidden unit has the rectified linear non-linearity. $h = \max(0, z)$

Output unit is linear. Loss function: $C = \frac{1}{2}(y - t)^2$.

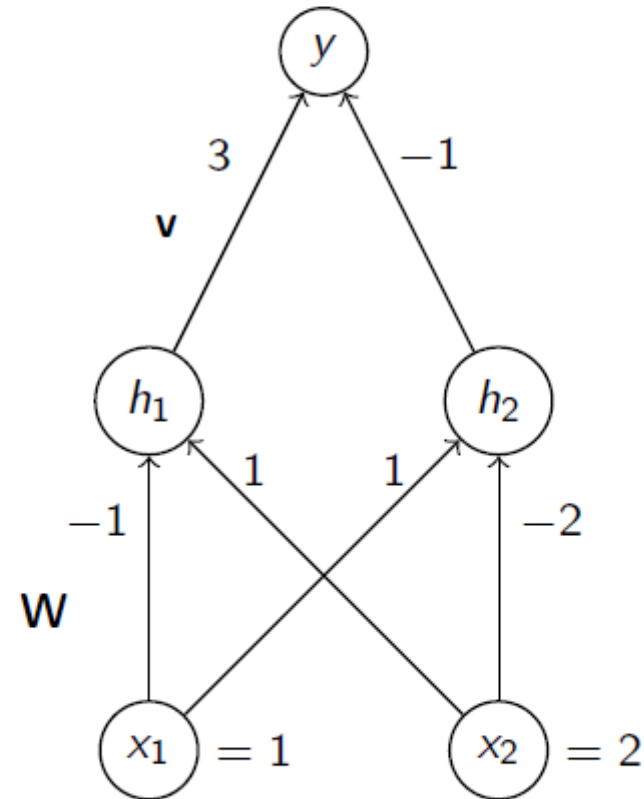
Compute the following -

The input to each hidden unit. $z_i = \sum_{j=1}^2 x_j W_{ij}$.

Output from each hidden unit. $h_i = \max(0, z_i)$.

Input to the output unit. $y = \sum_{i=1}^2 v_i h_i$.

Loss. $C = \frac{1}{2}(y - t)^2$.



Question 1: Backprop Example

Data : $x = (1, 2)$, $t = 1$.

Parameters : $\mathbf{W}$: 2×2 matrix, $\mathbf{v}$: 2-dimensional vector.

Each hidden unit has the rectified linear non-linearity. $h = \max(0, z)$

Output unit is linear. Loss function: $C = \frac{1}{2}(y - t)^2$.

Compute the following -

The input to each hidden unit. $z_i = \sum_{j=1}^2 x_j W_{ij}$.

Output from each hidden unit. $h_i = \max(0, z_i)$.

Input to the output unit. $y = \sum_{i=1}^2 v_i h_i$.

Loss. $C = \frac{1}{2}(y - t)^2$.

Derivative of loss w.r.t y : $\frac{\partial C}{\partial y} = y - t$.

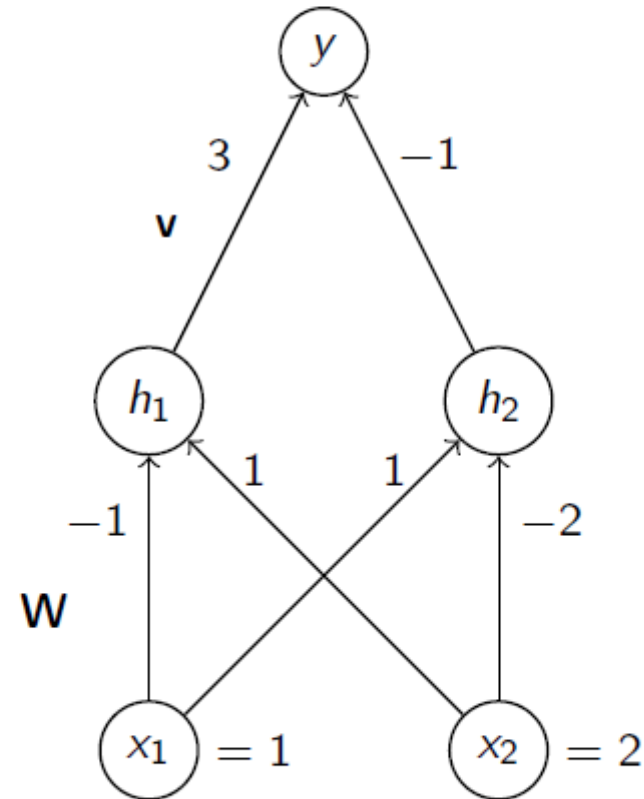
Derivative of loss w.r.t h_i : $\frac{\partial C}{\partial h_i} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial h_i}$.

Derivative of loss w.r.t z_i : $\frac{\partial C}{\partial z_i} = \frac{\partial C}{\partial h_i} \frac{\partial h_i}{\partial z_i}$.

Derivative of loss w.r.t x_j : $\frac{\partial C}{\partial x_j} = \sum_{i=1}^2 \frac{\partial C}{\partial z_i} \frac{\partial z_i}{\partial x_j}$.

Derivative of loss w.r.t W_{ij} : $\frac{\partial C}{\partial W_{ij}} = \frac{\partial C}{\partial z_i} \frac{\partial z_i}{\partial W_{ij}}$.

Derivative of loss w.r.t v_i : $\frac{\partial C}{\partial v_i} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial v_i}$.



Question 1: Backprop Example

Forward pass:

$$\begin{aligned}z_1 &= W_{11}x_1 + W_{12}x_2 \\&= -1 \cdot 1 + 1 \cdot 2 = 1\end{aligned}$$

$$\begin{aligned}z_2 &= W_{21}x_1 + W_{22}x_2 \\&= 1 \cdot 1 + -2 \cdot 2 = -3\end{aligned}$$

$$h_1 = \max(z_1, 0) = 1$$

$$h_2 = \max(z_2, 0) = 0$$

$$\begin{aligned}y &= v_1h_1 + v_2h_2 \\&= 3 \cdot 1 + -1 \cdot 0 = 3\end{aligned}$$

Loss and loss derivative:

$$C = \frac{1}{2}(y - t)^2 = 2$$

$$\frac{\partial C}{\partial y} = y - t = 2$$

Question 1: Backprop Example

Forward pass:

$$\begin{aligned}z_1 &= W_{11}x_1 + W_{12}x_2 \\ &= -1 \cdot 1 + 1 \cdot 2 = 1\end{aligned}$$

$$\begin{aligned}z_2 &= W_{21}x_1 + W_{22}x_2 \\ &= 1 \cdot 1 + -2 \cdot 2 = -3\end{aligned}$$

$$h_1 = \max(z_1, 0) = 1$$

$$h_2 = \max(z_2, 0) = 0$$

$$\begin{aligned}y &= v_1h_1 + v_2h_2 \\ &= 3 \cdot 1 + -1 \cdot 0 = 3\end{aligned}$$

Loss and loss derivative:

$$C = \frac{1}{2}(y - t)^2 = 2$$

$$\frac{\partial C}{\partial y} = y - t = 2$$

Activation derivatives:

$$\frac{\partial C}{\partial h_1} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial h_1}$$

$$\begin{aligned}&= \frac{\partial C}{\partial y} v_1 \\ &= 2 \cdot 3 = 6\end{aligned}$$

$$\frac{\partial C}{\partial h_2} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial h_2}$$

$$\begin{aligned}&= \frac{\partial C}{\partial y} v_2 \\ &= 2 \cdot -1 = -2\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial z_1} &= \frac{\partial C}{\partial h_1} \frac{\partial h_1}{\partial z_1} \\ &= 6 \cdot 1 = 6\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial z_2} &= \frac{\partial C}{\partial h_2} \frac{\partial h_2}{\partial z_2} \\ &= -2 \cdot 0 = 0\end{aligned}$$

$$\frac{\partial C}{\partial x_1} = \frac{\partial C}{\partial z_1} \frac{\partial z_1}{\partial x_1} + \frac{\partial C}{\partial z_2} \frac{\partial z_2}{\partial x_1}$$

$$= \frac{\partial C}{\partial z_1} W_{11} + \frac{\partial C}{\partial z_2} W_{21}$$

$$= 6 \cdot -1 + 0 \cdot 1 = -6$$

$$\frac{\partial C}{\partial x_2} = \frac{\partial C}{\partial z_1} \frac{\partial z_1}{\partial x_2} + \frac{\partial C}{\partial z_2} \frac{\partial z_2}{\partial x_2}$$

$$= \frac{\partial C}{\partial z_1} W_{12} + \frac{\partial C}{\partial z_2} W_{22}$$

$$= 6 \cdot 1 + 0 \cdot -2 = 6$$

Question 1: Backprop Example

Weight derivatives:

$$\begin{aligned}\frac{\partial C}{\partial v_1} &= \frac{\partial C}{\partial y} \frac{\partial y}{\partial v_1} \\ &= \frac{\partial C}{\partial y} h_1 \\ &= 2 \cdot 1 = 2\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial v_2} &= \frac{\partial C}{\partial y} \frac{\partial y}{\partial v_2} \\ &= \frac{\partial C}{\partial y} h_2 \\ &= 2 \cdot 0 = 0\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial W_{11}} &= \frac{\partial C}{\partial z_1} \frac{\partial z_1}{\partial W_{11}} \\ &= \frac{\partial C}{\partial z_1} x_1 \\ &= 6 \cdot 1 = 6\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial W_{12}} &= \frac{\partial C}{\partial z_1} \frac{\partial z_1}{\partial W_{12}} \\ &= \frac{\partial C}{\partial z_1} x_2 \\ &= 6 \cdot 2 = 12\end{aligned}$$

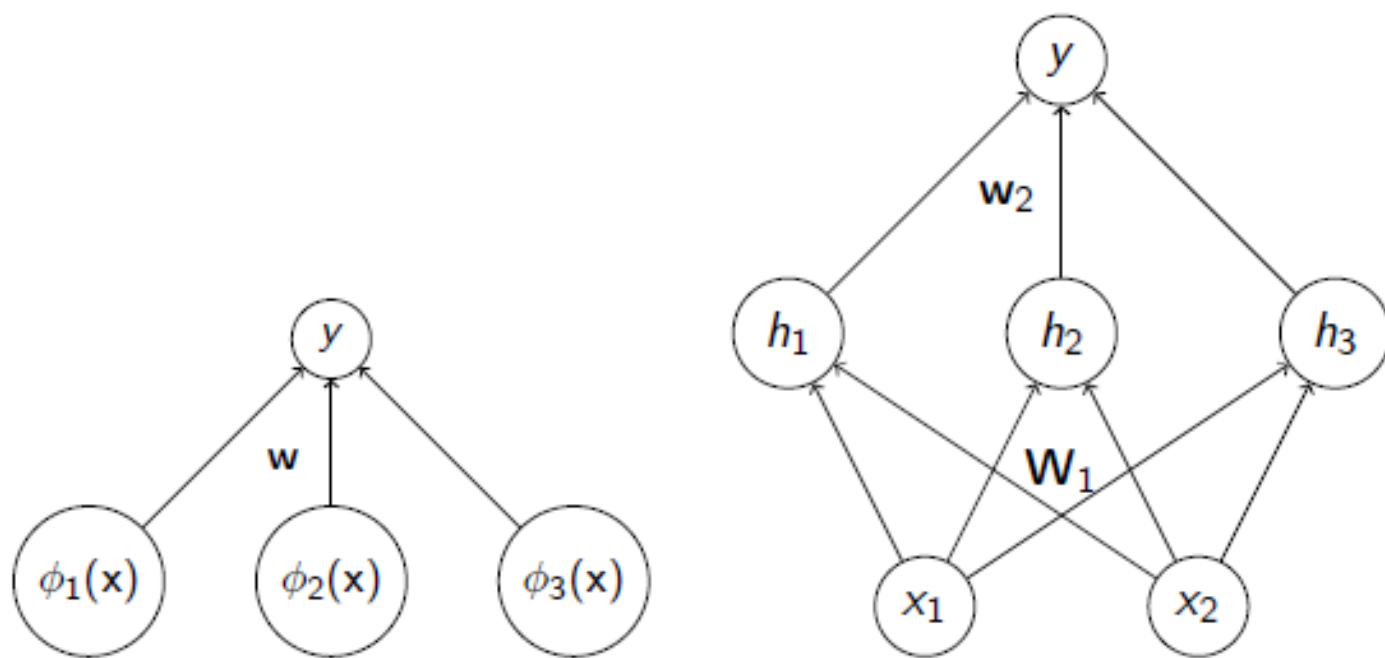
$$\begin{aligned}\frac{\partial C}{\partial W_{21}} &= \frac{\partial C}{\partial z_2} \frac{\partial z_2}{\partial W_{21}} \\ &= \frac{\partial C}{\partial z_2} x_1 \\ &= 0 \cdot 1 = 0\end{aligned}$$

$$\begin{aligned}\frac{\partial C}{\partial W_{22}} &= \frac{\partial C}{\partial z_2} \frac{\partial z_2}{\partial W_{22}} \\ &= \frac{\partial C}{\partial z_2} x_2 \\ &= 0 \cdot 2 = 0\end{aligned}$$

Why use a multi-layer network?

We want to **learn features** instead of hand-designing them.

We will see how **backpropagation** can be used to learn them.



An Example of a Neural Net

Network to classify handwritten digits.

